

ЛАБОРАТОРНЫЕ РАБОТЫ. ЧАСТЬ 1

ПО ДИСЦИПЛИНЕ ТЕХНОЛОГИИ РАЗРАБОТКИ
ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Е.Н. ЗЕРНОВА, Л.Д. ЧЕЛИЩЕВА

СОДЕРЖАНИЕ

Лабораторная работа №1 Анализ проблемы. Постановка задачи	3
Краткие теоретические сведения	3
Задание и порядок проведения работы	9
Лабораторная работа №2 Разработка пользовательских историй при проектировании ИС	10
Краткие теоретические сведения	10
Задание и порядок проведения работы	12
Лабораторная работа №3 Функциональные и нефункциональные требования	13
Краткие теоретические сведения	13
Задание и порядок проведения работы	20
Лабораторная работа №4 Описание бизнес-процессов	21
Краткие (ну, не совсем) теоретические сведения	21
Задание и порядок проведения работы	40
Лабораторная работа №5 Построение диаграммы вариантов использования	40
Краткие теоретические сведения	40
Задание и порядок проведения работы	43
Лабораторная работа №6 Построение диаграммы последовательности	43
Краткие теоретические сведения	43
Задание и порядок проведения работы	51
Лабораторная работа №7 Построение диаграммы активности	51
Краткие теоретические сведения	51
Задание и порядок проведения работы	56
Лабораторная работа №8 Проектирование пользовательского интерфейса	57
Краткие теоретические сведения	57
Задание и порядок проведения работы	60
Лабораторная работа №9 Проектирование БД	61
Краткие теоретические сведения	61
Задание и порядок проведения работы	66

ЛАБОРАТОРНАЯ РАБОТА №1

АНАЛИЗ ПРОБЛЕМЫ. ПОСТАНОВКА ЗАДАЧИ

Цель работы: сформировать навыки:

- работы с реальными заказчиками программных систем;
- идентификации заинтересованных лиц и интервью с ними;
- анализа полученного материала;
- формулирования проблемы, ее актуальности и потребностей заинтересованных лиц.

Краткие теоретические сведения

На этапе анализа проблемы проводится анализ предметной области, для которой разрабатывается ПО. Цели этапа:

- 1) определение границ, или контура, системы;
- 2) описание объектов автоматизации и/или формализации знаний об этих объектах;
- 3) выявление или определение потребностей заказчика ПО.

Анализ предметной области можно проводить, например, основываясь на теории системного анализа и использовать предложенные в ней методы.

Исходными данными для этапа системного анализа являются:

- 1) регламенты работы отделов и должностные инструкции сотрудников этих отделов;
- 2) анкеты опроса заинтересованных лиц;
- 3) записи интервью с заинтересованными лицами;
- 4) другие документы, имеющие отношение к исследуемому объекту.

Выходными данными, или результатом, этапа системного анализа являются:

- 1) перечень заинтересованных лиц;
- 2) список потребностей заинтересованных лиц в разрабатываемом ПО;
- 3) описание объектов автоматизации;
- 4) модель объектов автоматизации или предметной области.

Описание примера

Здесь формулируется задача, решением которой является разработка программного обеспечения.

Итак, в результате вступления России в Болонский процесс в РФ была инициирована реформа высшего профессионального образования, в соответствии с которой Министерством образования и науки РФ была разработана программа перевода традиционной системы оценки успеваемости студентов в систему зачетных единиц (кредитов). Это объясняется необходимостью унификации систем высшего образования с целью создания единого образовательного пространства в тех странах, которые уже вступили в Болонский процесс. В рамках этой программы все вузы страны должны к установленной дате перейти на новую систему. Красноярский государственный политехнический университет (КГПУ) начал решать поставленную перед ним задачу поэтапно. Одной из задач перехода на новую систему в КГПУ являлась автоматизация учета текущей успеваемости и промежуточных аттестаций студентов в целях унификации этого процесса на всех кафедрах и факультетах вуза, реализации возможности автоматизированного формирования отчетов, публикации на сайте вуза рейтингов успеваемости студентов.

Составление списка заинтересованных лиц

Заинтересованные лица – это все те, кто имеет прямое или косвенное отношение к процессу, автоматизация которого производится.

Для выявления заинтересованных лиц необходимо ответить на следующие вопросы:

- кто является пользователем системы?
- кто является заказчиком (покупателем) системы?
- на кого еще окажут влияние результаты работы системы?
- кто будет оценивать и принимать систему, когда она будет представлена и развернута?
- существуют ли другие внутренние или внешние пользователи системы, чьи потребности необходимо учесть?
- кто будет заниматься сопровождением новой системы?
- не забыли ли мы кого-нибудь?

В нашем примере определим будущих пользователей системы – это преподаватели, секретари кафедр и деканатов, заведующие кафедрами, системный администратор и сотрудники Учебного управления. Заказчиком нашей системы является вуз в лице первого проректора.

Теперь попытаемся выяснить, на кого еще будут оказывать влияние результаты работы нашей системы. Во-первых, на студентов, ведь рейтинги успеваемости всех

студентов будут доступны на сайте вуза. Во-вторых, по этой же причине, на родителей. В-третьих, на деканов факультетов и заместителей деканов по учебной работе, поскольку любые изменения в учебном процессе касаются их профессиональной деятельности.

Итак, рассмотрев первые три вопроса, мы практически охватили всех заинтересованных лиц. Поскольку сопровождать систему будет разработчик этой системы, то в список заинтересованных лиц мы его не включаем. Таким образом, получаем следующий список заинтересованных лиц для нашей системы:

- преподаватели;
- секретари кафедр;
- секретари деканатов;
- заведующие кафедрами;
- Учебное управление (в лице начальника);
- первый проректор;
- системный администратор (тот, кто будет администрировать нашу систему);
- студенты;
- родители студентов;
- заместители деканов по учебной работе;
- деканы факультетов.

Анкетирование и проведение интервью

Для выявления потребностей заказчика и описания объектов автоматизации можно проводить как анкетирование, так и интервью. Но наибольший эффект возможен только при проведении и того и другого.

Примеры анкеты и перечня вопросов для интервью приведены ниже.

Анкета для опроса заинтересованных лиц

1. Имя.
2. Наименование организации.
3. Наименование структурного подразделения.
4. Должность.
5. Кому Вы непосредственно подчиняетесь?
6. Каковы Ваши основные обязанности?
7. Что Вы в основном производите?
8. Для кого?

9. Какие документы или какую информацию можно считать входящими, или необходимыми, для Вашей деятельности?

10. Какие документы или какую информацию можно считать исходящими, или результатом Вашей деятельности?

11. Как измеряется успех Вашей деятельности?

12. Какие проблемы влияют на успешность Вашей деятельности?

13. Какие тенденции, если такие существуют, делают Вашу работу проще или сложнее?

14. Какой интерес или какие потребности у Вас есть относительно будущего решения (разрабатываемого ПО)?

Перечень вопросов для интервью

1) Оценка проблемы

a) Для каких проблем (прикладного типа) Вы ощущаете нехватку хороших решений? Назовите их. (Не забывайте спрашивать: «А еще?»)

2) По каждой проблеме выясняйте следующее:

a) почему существует эта проблема?

b) как она решается в настоящее время?

c) как заказчик (пользователь) хотел бы ее решать?

3) Понимание пользовательской среды

a) Каковы Ваши навыки в компьютерной области?

b) С какими типами приложений Вы имеете опыт работы?

c) Какая платформа используется?

d) Каковы Ваши планы относительно будущих платформ?

e) Используется ли ПО, которое имеет отношение к данной проблеме?

(Если да, то пусть о нем немного расскажут.)

f) Каковы Ваши ожидания относительно практичности продукта?

g) В каком виде должна быть представлена справочная информация для пользователя (в интерактивном или печатном)?

4) Резюме (перечисляются основные пункты, чтобы проверить, все ли правильно вы поняли)

a) Итак, Вы сказали мне... (перечислите описанные заказчиком проблемы своими словами)

b) Адекватно ли этот список представляет проблемы, которые имеются при существующем решении?

c) Какие еще проблемы Вы испытываете?

Заключение аналитика.

После интервью, пока его данные еще свежи в вашей памяти, зафиксируйте не менее трех потребностей или проблем с наивысшими приоритетами, выявленных вами в беседе с данным заказчиком (пользователем).

После проведения анкетирования и интервьюирования необходимо обработать собранную информацию. На основе этих данных нужно сформулировать перечень потребностей заказчиков, построить модель предметной области и описать объект/объекты автоматизации. Все эти результаты в дальнейшем будут использованы при написании технического задания (ТЗ) на разрабатываемую систему.

Интервью с начальником учебного управления

Вопрос	Ответ
1. Имя	Иванов Петр Владимирович
2. Наименование структурного подразделения	Учебное управление
3. Должность	Начальник Учебного управления
4. Кому Вы непосредственно подчиняетесь?	Первому проректору
5. Каковы Ваши основные обязанности?	Обеспечение общего управления учебной деятельностью. Обеспечение качества и своевременного выполнения возложенных на Учебное управление задач и функций. Обеспечение выполнения мероприятий по защите конфиденциальной и информационной безопасности в подразделениях управления
6. Что Вы в основном производите?	Функции, выполняемые Учебным управлением: 1. Создание единого нормативно-справочного поля учебного процесса. 2. Организационное сопровождение учебного процесса. 3. Планирование ресурсов учебного процесса (объемов работ кафедр, площадей, численности контингента, нагрузки на условиях почасовой оплаты, бланочной документации и др.). 4. Контроль за ходом организации и анализ показателей учебного процесса (как внешних, так и внутренних). 5. Ответность на различные уровни управления. 6. Анализ выполнения учебной нагрузки, трудоемкости отдельных параметров учебного процесса, результатов сессии, результатов государственных аттестационных комиссий. Разработка графика учебного процесса. 7. Составление расписания учебных занятий, экзаменов, зачетов и графика защит. 8. Контроль занятости аудиторного фонда, составления семестровых планов, проведения занятий, заседаний государственных аттестационных комиссий, хода сессий, делопроизводства факультетов, оформления приложений к дипломам. 9. Учет движения студентов по всем формам обучения (очное, очно-заочное и заочное, экстернат, второе высшее образование, обучение по сокращенным программам)
7. Какие документы или какую информацию можно считать входящими, или необходимыми, для Вашей деятельности?	1. Приказы и инструктивные письма Минобробразования России по учебно-методическим вопросам. 2. Положение об Учебном управлении. 3. Должностные инструкции сотрудников. 4. Приказы ректора по контингенту студентов (первые экземпляры). 5. Годовые планы приема абитуриентов. 6. Планы работы факультетов.

	7. Отчеты председателей государственных аттестационных комиссий по всем специальностям и направлениям
8. Какие документы или какую информацию можно считать исходящими, или результатом Вашей деятельности?	1. Семестровые планы занятий. 2. Расписание учебных занятий. 3. Расписание экзаменов. 4. Отчеты вуза по учебно-методической работе за учебный год. 5. Сводные статистические отчеты вуза о движении контингента студентов на начало и конец учебного года
9. Как измеряется успех Вашей деятельности?	В настоящее время отсутствуют количественные показатели оценки деятельности управления
10. Какие проблемы влияют на успешность Вашей деятельности?	Проблемы, связанные с организацией учебного процесса. В настоящее время основной проблемой является перевод учебного процесса в систему зачетных единиц
11. Какой интерес или какие потребности у Вас есть относительно будущего решения (разрабатываемого ПО)?	Разрабатываемая система должна быть максимально эргономичной, работать стабильно (без сбоев); отклик системы не должен вызывать у пользователей раздражения; реализуемая функциональность должна полностью удовлетворить потребности пользователя

В качестве примера проведем анкетирование и интервью с начальником Учебного управления. Для этого воспользуемся одним перечнем вопросов, совместив две эти процедуры воедино. По сути, мы проведем только интервью. Пример результата интервью (вопросы и ответы) приведен в таблице.

Список потребностей заинтересованных лиц

В результате анкетирования и интервьюирования всех заинтересованных лиц были сформулированы потребности заказчика относительно разрабатываемого ПО. Далее необходимо провести аналогию между выявленными потребностями и структурой и требованиями ТЗ в соответствии с ГОСТ 34.602–89. Таким образом, потребности заказчика в ТЗ могут быть описаны в разделе «Назначение и цели создания системы».

В нашем примере были выявлены следующие потребности:

- 1) унифицировать процесс оценивания знаний в системе кредитов на всех кафедрах и факультетах вуза;
- 2) минимизировать субъективность при оценивании студентов в промежуточных аттестациях;
- 3) реализовать возможность автоматического формирования рейтингов студентов по разным параметрам в системе кредитов;
- 4) реализовать возможность формирования единой отчетности на кафедрах и факультетах.

Задание и порядок проведения работы

- 1) Составить перечень заинтересованных лиц.

- 2) Провести интервью и/или анкетирование с каждым заинтересованным лицом.
- 3) Проанализировать полученную информацию и сформулировать актуальность проблемы и потребности заинтересованных лиц.

Контрольные вопросы

- 1) Что является исходными данными для анализа проблемы (предметной области)?
- 2) Что является результатом этапа системного анализа предметной области?
- 3) Как определить заинтересованных лиц?
- 4) Какой метод сбора информации наиболее эффективен?
- 5) Для чего проводятся интервьюирование и анкетирование?

ЛАБОРАТОРНАЯ РАБОТА №2

РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКИХ ИСТОРИЙ ПРИ ПРОЕКТИРОВАНИИ ИС

Цель работы: освоение технологии разработки и управления требованиями

Краткие теоретические сведения

В практике подготовки требований, которые в дальнейшем будут составлять каркас разрабатываемого программного продукта, принято использовать один из стандартизированных подходов для их описания – User Stories ("пользовательские истории").

Пользовательские истории формулируются как одно или более предложений на "повседневном" языке пользователя. Они получают относительно небольшие по объему, что удобно как для составления, так и для их обсуждения, приоритизации, планирования, оценки и дальнейшей работы с ними. Пользовательские истории получают в виде алгоритма действий пользователя с реализуемым программным продуктом. Адекватная оценка трудоемкости историй позволяет планировать сроки ее реализации, тем самым управляя содержанием спринтов. Разработать оптимальную пользовательскую историю не так просто, нужен определенный навык, который позволит создать качественное описание требований пользователей к реализуемому программному продукту, но в награду за это получают следующие преимущества:

- Краткость. Пользовательская история описывает небольшую часть бизнес-

ценности, которую возможно реализовать за период спринта.

- "Незатратность" создания и сопровождения. За счет своей "компактности" пользовательские требования достаточно просто создать и сопровождать их изменения на всем протяжении жизненного цикла продукта.

- Вовлечение ключевых пользователей в процесс создания продукта. За счет своей доступности бизнес-требования смогут стать реальным "мостиком" между пользователями и командой разработчиков. Это позволит более адекватно управлять ожиданиями пользователей и вовлечь их на нужную степень погружения в процесс разработки продукта.

- Облегчают оценку заданий. Формат пользовательских историй способствует более точной оценке необходимых системных разработок/доработок.

Чем более компактный объем имеет пользовательская история, тем проще выполнять ее оценку. Это приводит к более верным оценкам сроков реализации программного продукта и к более понятному планированию выполняемых работ. При этом важно знать меру в уменьшении и дроблении Use cases. Если сделать их слишком много, то у вас получится огромный список задач в бэклоге продукта, и процесс управления ими и фиксации на карте историй усложнится.

Для пользователей требования, выраженные в виде пользовательских историй, являются основным инструментом влияния на разрабатываемый программный продукт. Пользовательские истории определяют формат, в котором у пользователей есть возможность отразить все те важные факторы, которые, по их мнению, должны быть учтены в процессе автоматизации.

Важно отдавать себе отчет в том, что пользовательские истории не обеспечивают полноту всех функциональных требований и имеют ряд недостатков:

- Не масштабируются для больших программных продуктов. Пользовательские истории хорошо себя зарекомендовали, когда речь идет о создании небольших или средних по объемам и сложности программным продуктам. За счет того, что этот вид работы с требованиями ориентирован на непосредственную работу с пользователями и поддерживается "бизнес-лексиконом", он неприменим в работе над крупными информационными системами, когда на первый план выходит организационная структура проекта/процесса, и важны формальные признаки сдачи/приемки работ.

- Требовательны к квалификации разработчиков. Разработчики, работающие с пользовательскими историями, должны обладать высокой квалификацией и неплохими коммуникативными навыками, которые позволят им получить необходимые уточнения уже

изложенных требований. Как правило, таких разработчиков немного. И это еще один неоспоримый недостаток.

- Не являются средством документирования. Пользовательские истории – это небольшое и удобное представление информации. Они сформулированы на ежедневном языке пользователя и содержат небольшие детали, оставаясь открытыми для интерпретации. Они помогают понимать, что должна делать система, но при этом пользовательских требований недостаточно, чтобы понять, как будет организована логика системы. Пользовательские требования являются необходимой "верхушкой" для понимания назначения информационной системы, но для реализации системы разработчику приходится додумывать множество значимых деталей.

Инженерия требований – важный этап в создании каждого программного продукта. Функциональные обязанности по извлечению, разработке и управлению требованиями ложатся на всех членов команды, именно поэтому в процессе работы с ними задействованы такие артефакты, как пользовательские истории, назначение которых – облегчить получение информации, необходимой для качественной реализации программного продукта.

Формат пользовательской истории как правило следующий:

Я как тип пользователя, хочу действие, для того чтобы цель.

Пример:

Я как покупатель, хочу сформировать отчет по покупкам, для того чтобы проверить свои покупки.

Придерживаться такой структуры необязательно, но она помогает определить критерии готовности работы. История выполнена, когда упомянутый тип пользователя получает требуемую ценность. В идеале, команды формулируют свою собственную структуру и придерживаются ее.

Задание и порядок проведения работы

Ознакомиться с теоретическим материалом, разработать не менее 16 пользовательских историй от различных типов пользователей.

ЛАБОРАТОРНАЯ РАБОТА №3

ФУНКЦИОНАЛЬНЫЕ И НЕФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

Краткие теоретические сведения

Требования к программной системе часто классифицируются как функциональные, нефункциональные и требования предметной области.

Функциональные требования задают “что” система должна делать; **нефункциональные** – с соблюдением “каких условий” (например, скорость отклика при выполнении заданной операции); часто функциональные требования представляют в виде сценариев (вариантов использования) Use Case.

1. **Функциональные требования.** Это перечень сервисов, которые должна выполнять система, причем должно быть указано, как система реагирует на те или иные входные данные, как она ведет себя в определенных ситуациях и т.д. В некоторых случаях указывается, что система не должна делать.

2. **Нефункциональные требования.** Описывают характеристики системы и ее окружения, а не поведение системы. Здесь также может быть приведен перечень ограничений, накладываемых на действия и функции, выполняемые системой. Они включают временные ограничения, ограничения на процесс разработки системы, стандарты и т.д.

3. **Требования предметной области.** Характеризуют ту предметную область, где будет эксплуатироваться система. Эти требования могут быть функциональными и нефункциональными.

В действительности четкой границы между этими типами требований не существует. Например, пользовательские требования, касающиеся безопасности системы, можно отнести к нефункциональным. Однако при более детальном рассмотрении такое требование можно отнести к функциональным, поскольку оно порождает необходимость включения в систему средства авторизации пользователя. Поэтому, рассматривая далее эти виды требований, мы должны всегда помнить, что данная классификация в значительной степени искусственна.

Классический пример (см. рисунок 4.3) высокоуровневого структурирования групп требований как требований к продукту описан в работах одного из классиков дисциплины управления требованиями – Карла Вигерса.

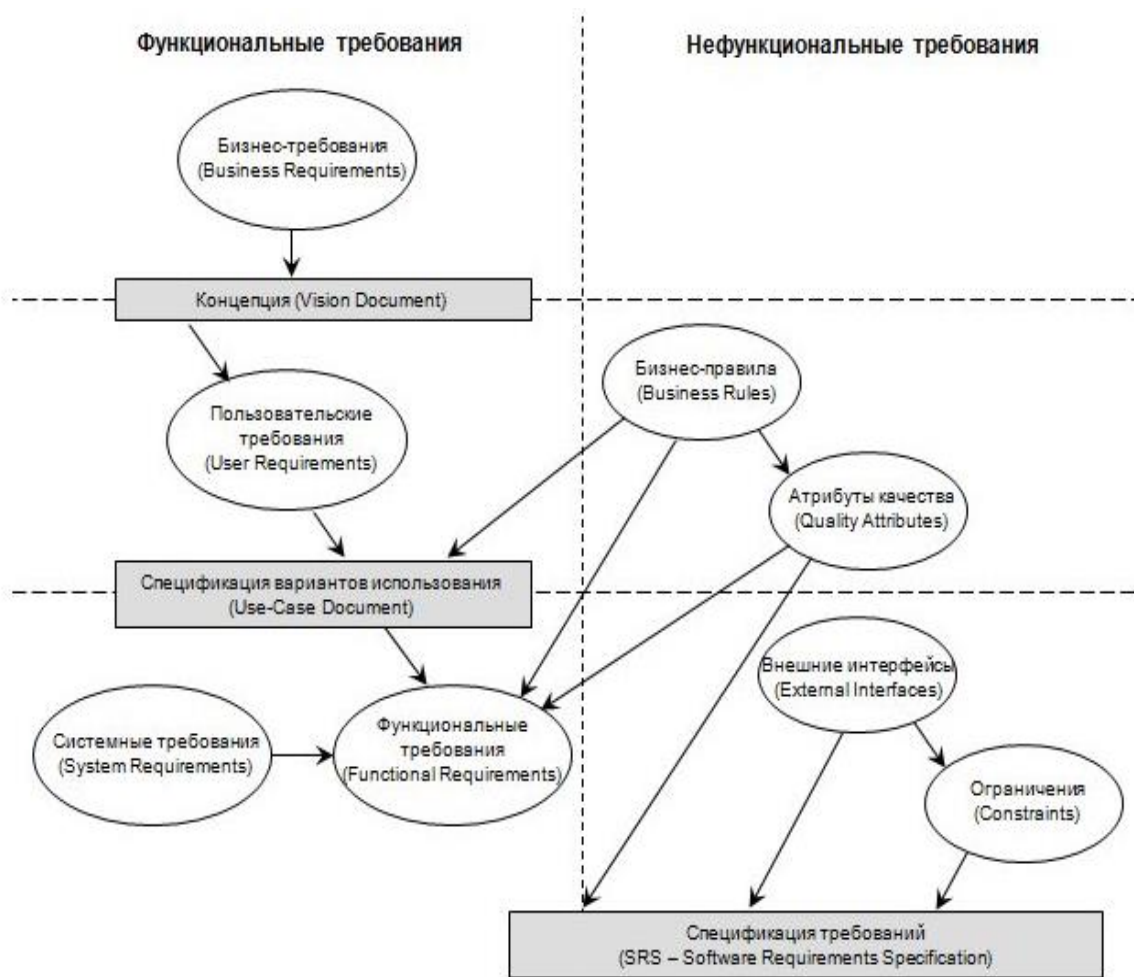


Рисунок 4.3. Уровни требований по Вигерсу

- Группа функциональных требований
 - Бизнес-требования (Business Requirements) – определяют высокоуровневые цели организации или клиента (потребителя) – заказчика разрабатываемого программного обеспечения.
 - Пользовательские требования (User Requirements) – описывают цели/задачи пользователей системы, которые должны достигаться/выполняться пользователями при помощи создаваемой программной системы. Эти требования часто представляют в виде вариантов использования (Use Cases).
 - Функциональные требования (Functional Requirements) – определяют функциональность (поведение) программной системы, которая должна быть создана разработчиками для предоставления возможности выполнения пользователями своих обязанностей в рамках бизнес-требований и в контексте пользовательских требований.
- Группа нефункциональных требований (Non-Functional Requirements)
 - Бизнес-правила (Business Rules) – включают или связаны с корпоративными регламентами, политиками, стандартами, законодательными актами, внутрикорпоративными инициативами (например, стремление достичь зрелости процессов

по СММІ 4-го уровня), учетными практиками, алгоритмами вычислений и т.д. На самом деле, достаточно часто можно видеть недостаточное внимание такого рода требованиям со стороны сотрудников ИТ-департаментов и, в частности, технических специалистов, вовлеченных в проект. Business Rules Group дает понимание бизнес-правила, как “положения, которые определяют или ограничивают некоторые аспекты бизнеса. Они подразумевают организацию структуры бизнеса, контролируют или влияют на поведение бизнеса”. Бизнес-правила часто определяют распределение ответственности в системе, отвечая на вопрос “кто будет осуществлять конкретный вариант, сценарий использования” или диктуют появление некоторых функциональных требований. В контексте дисциплины управления проектами (уже вне проекта разработки программного обеспечения, но выполнения бизнес-проектов и бизнес-процессов) такие правила обладают высокой значимостью и, именно они, часто определяют ограничения бизнес-проектов, для автоматизации которых создается соответствующее программное обеспечение.

- Внешние интерфейсы (External Interfaces) – часто подменяются “пользовательским интерфейсом”. На самом деле вопросы организации пользовательского интерфейса безусловно важны в данной категории требований, однако, конкретизация аспектов взаимодействия с другими системами, операционной средой (например, запись в журнал событий операционной системы), возможностями мониторинга при эксплуатации – все это не столько функциональные требования (к которым ошибочно приписывают иногда такие характеристики), сколько вопросы интерфейсов, так как функциональные требования связаны непосредственно с функциональностью системы, направленной на решение бизнес-потребностей.

- Атрибуты качества (Quality Attributes) – описывают дополнительные характеристики продукта в различных “измерениях”, важных для пользователей и/или разработчиков. Атрибуты касаются вопросов портируемости, интероперабельности (прозрачности взаимодействия с другими системами), целостности, устойчивости и т.п.

- Ограничения (Constraints) – формулировки условий, модифицирующих требования или наборы требований, сужая выбор возможных решений по их реализации. В частности, к ним могут относиться параметры производительности, влияющие на выбор платформы реализации и/или развертывания (протоколы, серверы приложений, баз данных, ...), которые, в свою очередь, могут относиться, например, к внешним интерфейсам.

- Системные требования (System Requirements) – иногда классифицируются как составная часть группы функциональных требований (не путайте с как таковыми “функциональными требованиями”). Описывают высокоуровневые требования к программному обеспечению, содержащему несколько или много взаимосвязанных

подсистем и приложений. При этом, система может быть как целиком программной, так и состоять из программной и аппаратной частей. В общем случае, частью системы может быть персонал, выполняющий определенные функции системы, например, авторизация выполнения определенных операций с использованием программно-аппаратных подсистем.

Функциональные требования

Эти требования описывают поведение системы и сервисы (функции), которые она выполняет, и зависят от типа разрабатываемой системы и от потребностей пользователей. Если функциональные требования оформлены как пользовательские, они, как правило, описывают системы в обобщенном виде. В противоположность этому функциональные требования, оформленные как системные, описывают систему максимально подробно, включая ее входные и выходные данные, исключения и т.д.

Функциональные требования для программных систем могут быть описаны разными способами. Рассмотрим для примера функциональные требования к библиотечной системе университета, предназначенной для заказа книг и документов из других библиотек.

1. Пользователь должен иметь возможность проводить поиск необходимых ему книг и документов или по всему множеству доступных каталожных баз данных или по определенному их подмножеству.

2. Система должна предоставлять пользователю подходящее средство просмотра библиотечных документов.

3. Каждый заказ должен быть снабжен уникальным идентификатором (ORDERJD), который копируется в формуляр пользователя для постоянного хранения.

Эти функциональные пользовательские требования определяют свойства, которыми должна обладать система. Они взяты из документа, содержащего пользовательские требования, и показывают, что функциональные требования могут быть описаны с разным уровнем детализации (сравните первое и третье требования).

Многие проблемы, возникающие при разработке систем, связаны с неточностью и "размытостью" спецификации требований. Естественно, разработчики интерпретируют требования, допускающие двоякое толкование, так, чтобы систему было проще реализовать. Но это толкование может не совпадать с ожиданиями заказчика. Такая ситуация приводит к разработке новых требований и внесению изменений в систему. Это, в свою очередь, ведет к задержке сдачи готовой системы и ее удорожанию.

Рассмотрим второе требование к библиотечной системе из приведенного выше списка и обратим внимание на выражение "подходящее средство просмотра документов". Библиотечная система может предоставлять документы в широком спектре форматов. В

требовании подразумевается, что система должна предоставить средства для просмотра документов в любом формате. Но поскольку это условие четко не выписано, разработчики в случае дефицита времени могут использовать простое средство для просмотра текстовых документов и настаивать на том, что именно такое решение следует из данного требования.

В принципе спецификация функциональных требований должна быть комплексной и непротиворечивой. Комплексность подразумевает описание (определение) всех системных сервисов. Непротиворечивость означает отсутствие несовместимых и взаимоисключающих определений сервисов. На практике для больших и сложных систем крайне трудно разработать комплексную и непротиворечивую спецификацию функциональных требований. Причина кроется частично в сложности самой разрабатываемой системы, а частично – в несогласованных опорных точках зрения на то, что должна делать система. Эта несогласованность может не проявиться на этапе первоначального формулирования требований – для ее выявления необходим более глубокий анализ спецификации. Когда несогласованность системных функций проявится на каком-либо этапе жизненного цикла программы, в системную спецификацию придется внести соответствующие изменения.

Нефункциональные требования

Как следует из названия, нефункциональные требования не связаны непосредственно с функциями, выполняемыми системой. Они связаны с такими интеграционными свойствами системы, как надежность, время ответа или размер системы. Кроме того, нефункциональные требования могут определять ограничения на систему, например на пропускную способность устройств ввода-вывода, или форматы данных, используемых в системном интерфейсе.

Многие нефункциональные требования относятся к системе в целом, а не к отдельным ее средствам. Это означает, что они более значимы и критичны, чем отдельные функциональные требования. Ошибка, допущенная в функциональном требовании, может снизить качество системы, ошибка в нефункциональных требованиях может сделать систему неработоспособной.

Вместе с тем нефункциональные требования могут относиться не только к самой программной системе: одни могут относиться к технологическому процессу создания ПО, другие – содержать перечень стандартов качества, накладываемых на процесс разработки. Кроме того, в спецификации нефункциональных требований может быть указано, что проектирование системы должно выполняться только определенными CASE-средствами, и приведено описание процесса проектирования, которому необходимо следовать.

Нефункциональные требования отображают пользовательские потребности; при

этом они основываются на бюджетных ограничениях, учитывают организационные возможности компании-разработчика и возможность взаимодействия разрабатываемой системы с другими программными и вычислительными системами, а также такие внешние факторы, как правила техники безопасности, законодательство о защите интеллектуальной собственности и т.п. На рис. 4.4 показана классификация нефункциональных требований.

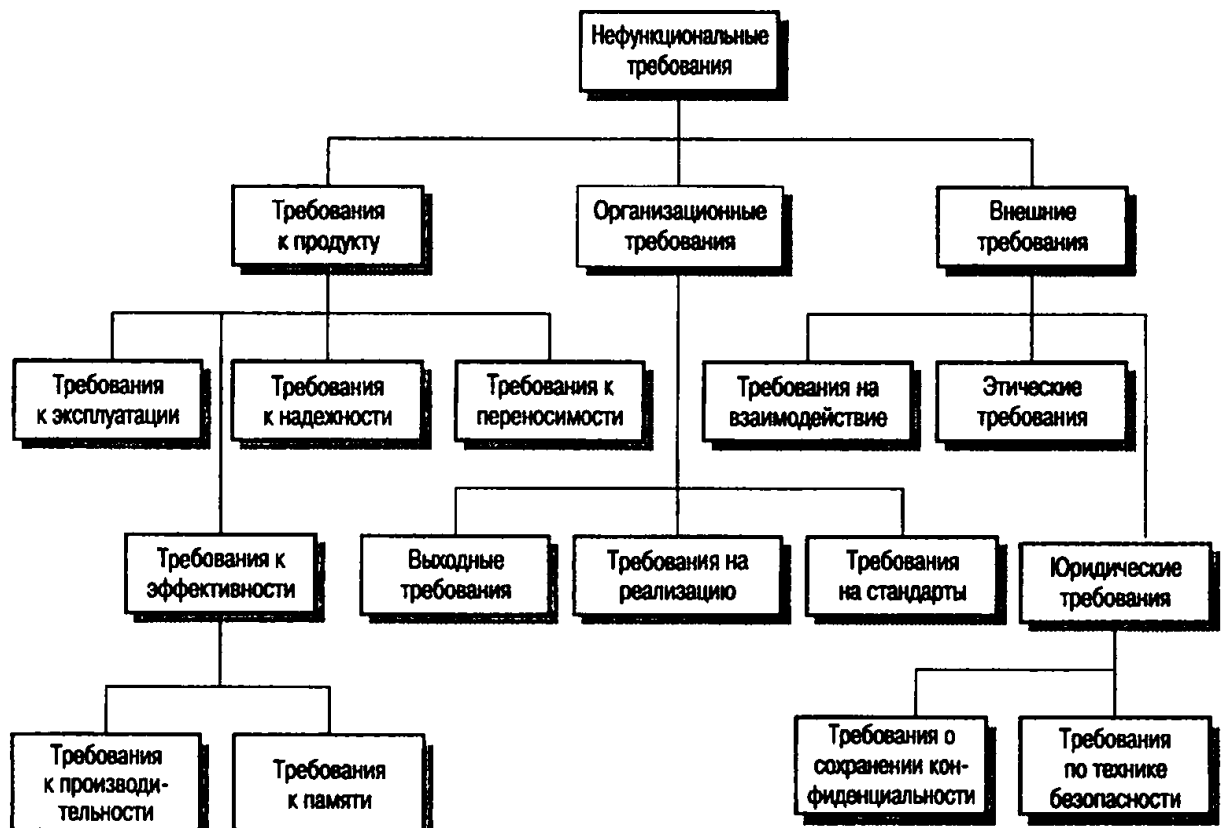


Рис. 4.4. Типы нефункциональных требований

Все нефункциональные требования, показанные на рис.4.4. разбиты на три большие группы.

1. Требования к продукту. Описывают эксплуатационные свойства программного продукта. Сюда относятся требования к производительности системы, объему необходимой памяти, надежности (определяет частоту возможных сбоев в системе), переносимости системы на разные компьютерные платформы и удобству эксплуатации.

2. Организационные требования. Отображают политику и организационные процедуры заказчика и разработчика ПО. Они включают стандарты разработки программного продукта, требования к реализации ПО (т.е. к языку программирования и методам проектирования), выходные требования, которые определяют сроки изготовления программного продукта, и сопутствующую документацию.

3. Внешние требования. Учитывают факторы, внешние по отношению к разрабатываемой системе и процессу ее разработки. Они включают требования,

определяющие взаимодействие данной системы с другими системами, юридические требования, следование которым гарантирует, что система будет разрабатываться и функционировать в рамках существующего законодательства, а также этические требования. Последние должны гарантировать, что система будет приемлемой для пользователей или заказчика.

Основная проблема нефункциональных требований состоит в том, что их выполнение трудно проверить. Часто они пишутся для того, чтобы отобразить общие цели заказчика системы, такие, как простота эксплуатации, возможность восстановления после сбоев или быстрый ответ на запросы пользователя. Реализация подобных требований может оказаться сложной для системных разработчиков, поскольку они нечетко сформулированы и открывают простор для различных толкований. Подобную ситуацию иллюстрирует пример 2. Здесь одним из основных показателей (целей) системы указана простота эксплуатации, что в виде нефункциональных требований можно выразить различными способами. В данном случае требование сформулировано так, что его можно проверить.

Пример 2. Системные цели и проверка требований.

Системная цель.

Система должна быть простой, в эксплуатации для опытного оператора и сводить количество его ошибок к минимуму.

Проверяемое нефункциональное требование.

Опытному оператору должны быть доступны все системные функции после двух часов обучения работе с данной системой. После такого обучения число ошибок оператора не должно превышать двух за рабочий день.

В идеале нефункциональные требования должны выражаться через количественные показатели, которые можно объективно измерить. В табл. 4.1 приведены показатели, с помощью которых можно специфицировать нефункциональные системные свойства. На практике выразить нефункциональные требования с помощью количественных показателей весьма затруднительно. Часто заказчик ПО не может оформить свое видение будущей системы посредством требований, выраженных количественными показателями. Либо некоторые системные требования, например удобство сопровождения, вообще нельзя выразить через количественные показатели. Кроме того, затраты на объективное измерение количественных нефункциональных требований могут оказаться крайне высокими.

Нефункциональные требования часто вступают в конфликт с другими требованиями, предъявляемыми системе. Например, в соответствии с одним из системных требований размер системы не должен превышать 4 Мбайт, поскольку она должна полностью поместиться в постоянное запоминающее устройство ограниченной емкости.

Другое требование обязывает использовать для написания системы язык программирования Ada, который часто применяется для создания критических систем реального времени. Но, допустим, откомпилированная системная программа, написанная на языке Ada, занимает более 4 Мбайт. Итак, одновременное выполнение этих требований невозможно. В этой ситуации следует отказаться от одного из требований. Можно или применить другой язык программирования, или увеличить объем памяти, выделяемый для системы.

Таблица 4.1. Количественные показатели нефункциональных требований

Показатель	Единицы измерения
Скорость	Количество выполненных транзакций в секунду;
	время реакции на действия пользователя;
	время обновления экрана
Размер	Килобайты;
	количество модулей памяти
Простота эксплуатации	Время обучения персонала;
	количество статей в справочной системе
Надежность	Средняя продолжительность времени между двумя последовательными проявлениями ошибок в системе;
	вероятность выхода системы из строя;
	коэффициент готовности системы
Устойчивость к сбоям	Время восстановления системы после сбоя; процент событий, приводящих к сбоям; вероятность порчи данных при сбоях
Переносимость	Процент машинно-зависимых операторов;
	количество машинно-зависимых подсистем

Задание и порядок проведения работы

Составить полные списки функциональных и нефункциональных требований к вашей системе.

ЛАБОРАТОРНАЯ РАБОТА №4

ОПИСАНИЕ БИЗНЕС-ПРОЦЕССОВ В НОТАЦИИ BPMN

Нотация BPMN (Business Process Modeling Notation) нужна для подробного описания логики выполнения бизнес-процесса, в том числе для отражения деталей процессов, таких как: события, исполнители каждого из действий, используемые и создаваемые документы и другие объекты, использующиеся в качестве входных данных для тех или иных действий или создающиеся в результате их выполнения.

В зависимости от целей построения BPMN-диаграмм, различают 3 уровня моделирования:

1. **Описательное моделирование**, когда нужно показать успешный путь выполнения бизнес-процесса, например, чтобы согласовать его с бизнес-пользователем. Здесь применяются самые простые элементы нотации, а сама диаграмма намеренно максимально упрощается.
2. **Аналитическое моделирование** используется, когда нужно полностью показать все варианты выполнения бизнес-процесса, включая логические ветвления и альтернативы. Такая диаграмма обычно создается для опытных пользователей и бизнес-аналитиков с помощью расширенного алфавита нотации, включая не только ее базовые самые простые элементы, но и более сложные.
3. **Исполняемое моделирование** предназначено для запуска на исполнение в BPMS-движке, чтобы создать веб-приложение. Здесь может использоваться всё многообразие алфавита этой нотации, включая добавление специальных параметров и скриптов, создаваемых разработчиками.

Алфавит нотации.

Главными объектами на диаграмме являются события и действия (задачи), которые соединяются потоком управления.

Поток управления — это последовательность шагов бизнес-процесса, в которой он выполняется.

Событие — это некий свершившийся факт, что-то, что возникает по ходу процесса или происходит в результате выполнения тех или иных действий. Например, «от клиента поступила заявка», «прошла неделя с момента подачи заявления» и т. д. Процесс в BPMN-диаграмме всегда начинается с события и должен заканчиваться событием.

Приведен базовый набор элементов, использующийся для отображения событий. Если внутри круга, изображающего события, вписан какой-то элемент, он называется **триггер**.

Тип события	Смысл триггера	Стартовое	Финишное
Простое	Начало или окончание процесса или подпроцесса		
Сообщение	Получение или отправка сообщения (данных в любом виде)		
Таймер	Наступление определённого времени, истечение временного интервала, временная цикличность		
Терминальное	Немедленное прекращение всех активных потоков бизнес-процесса («убивание» токенов)		

Триггер определяет тип и смысл события. Например, триггер в виде конверта означает, что пришли какие-то данные, причем совсем не обязательно в виде сообщения электронной почты. Триггер в виде часов связан со временем. Если событие имеет триггер, значит, поток управления двинется дальше только тогда, когда сработает триггер этого события. Например, получены данные, наступил определённый временной интервал и так далее.

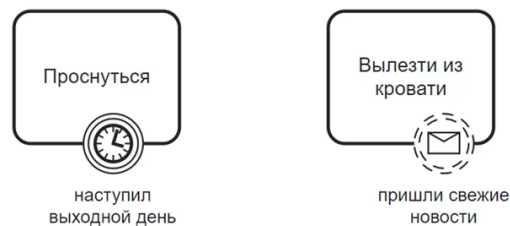
Действия.

Различают несколько типов действий.

 Получить приказ	Ожидание сообщения от другого участника или процесса. Эта задача приостанавливает выполнение процесса до тех пор, пока не будет получено сообщение. После получения сообщения задача завершается и считается выполненной.
 Выбрать отчет	Выполняется пользователем информационной системы, например, заказать пиццу. Используется для ввода данных, создания записей, выгрузки отчётов, манипуляции с объектами.
 Закрепить деталь	Выполняется человеком вручную без использования информационных систем, например, съесть пиццу

 Отобразить форму	Выполняется программным сервисом или аппаратным оборудованием автоматически без участия человека. Используется, чтобы показать логику взаимодействия информационных систем с пользователем.
 Сформировать отчет	Запускает скрипт, который выполняется автоматически той же информационной системой, которая управляет всем бизнес-процессом, например, BPMS, СЭД, ERP и т.д. Задача содержит одно или несколько действий, которые необходимо выполнить все для её завершения.

События могут располагаться в потоке управления между действиями процесса или на границе действия — в этом случае они считаются **граничными**.

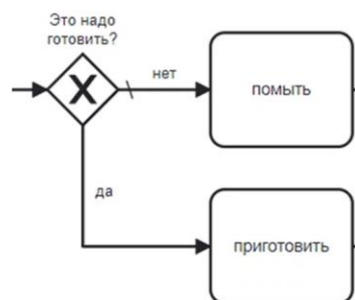


Граничные события являются промежуточными, они находятся на границе действия, обозначая те факты, которые случились при его выполнении. Они могут прерывать процесс (граничные прерывающие события) или активировать дополнительный поток управления, который выполняется одновременно с выполнением подпроцесса (граничные не прерывающие события). Граничные прерывающие события обозначается кругом с двойной сплошной окантовкой. У граничных непрерывающих событий окантовка тоже двойная, но в виде штриховой линии.

Логические операторы

Поскольку BPMN показывает логику выполнения бизнес-процесса, в диаграммах используются логические операторы, которые также называются развилками или шлюзами. Изначально их всего три: OR, XOR и AND.

XOR представляет собой исключающее или, когда только одна ветка из входящих или исходящих потоков может быть истинной.



В отличие от исключающего или, простое ИЛИ (OR) допускает возможность активации как нескольких веток, так и одной из них. В математическом смысле этот оператор реализует дизъюнкцию или логическое сложение переменных, что показано в таблице истинности на слайде.

Наконец, логическое И (AND) означает активацию всех входящих или исходящих в этот оператор потоков управления, реализуя логическое умножение переменных, т. е. операцию конъюнкции.



Дизъюнкция логическое ИЛИ (OR) ✓

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

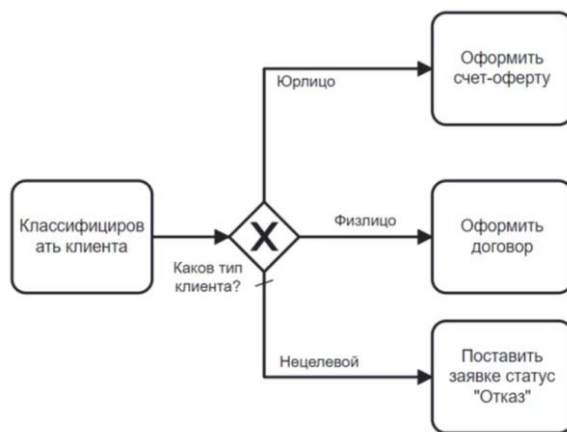
$X \vee Y = F$
 $X + Y = F$

Конъюнкция логическое И (AND) ^

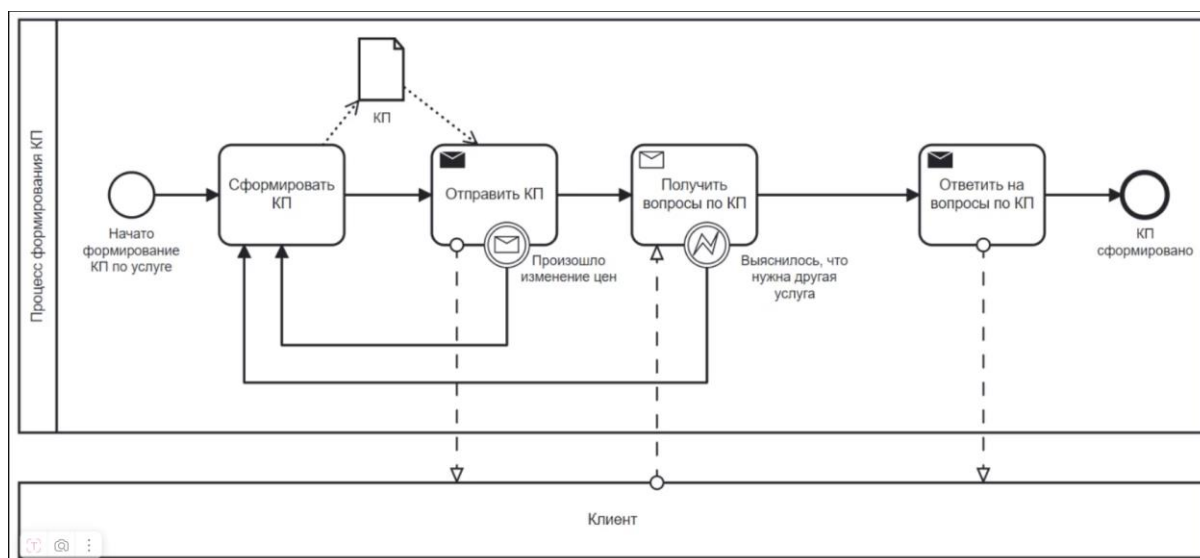
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

$X \wedge Y = F$
 $X \bullet Y = F$

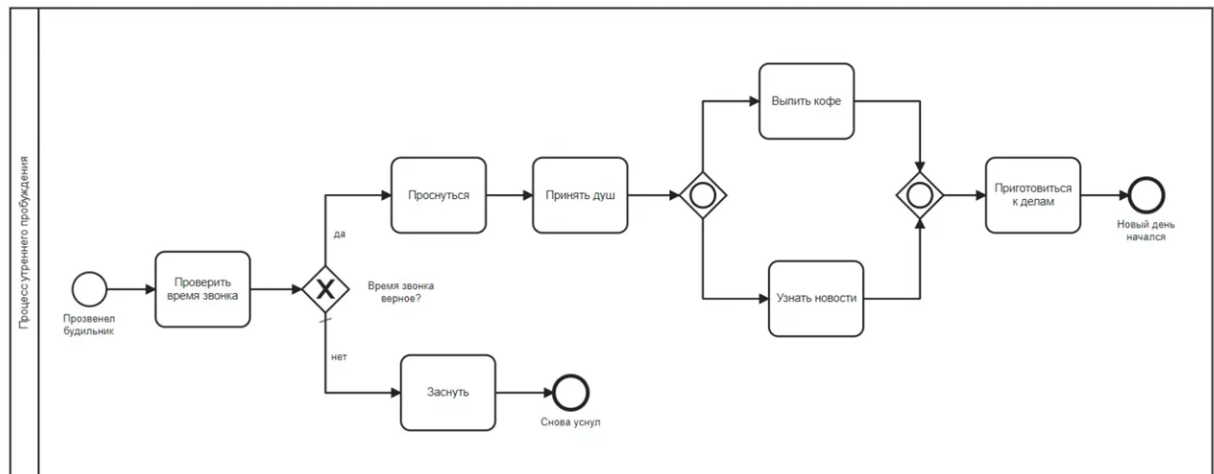
Если в диаграмме используются операторы обычного XOR, проверяющего условия по данным, и OR (неисключающего ИЛИ) рекомендуется пометить **поток по умолчанию**, который активируется, если другие условия не сработали. Поток по умолчанию допустимо не подписывать, если подписаны остальные потоки, и диаграмма остается понятной. В примере ниже «Нецелевой» — поток по умолчанию.



Но если в рамках отправки или получения сообщений произошли какие-то события, например, связанные со временем, это можно показать только с помощью действий, поскольку они допускают размещение граничных событий. Например, при отправке КП пришли данные о том, что цены услуги изменились и поэтому нужно сформировать КП заново. А во время получения вопросов по КП стало ясно, что клиенту нужна другая услуга, т. е. текущее КП неактуально и нужно сформировать новое.



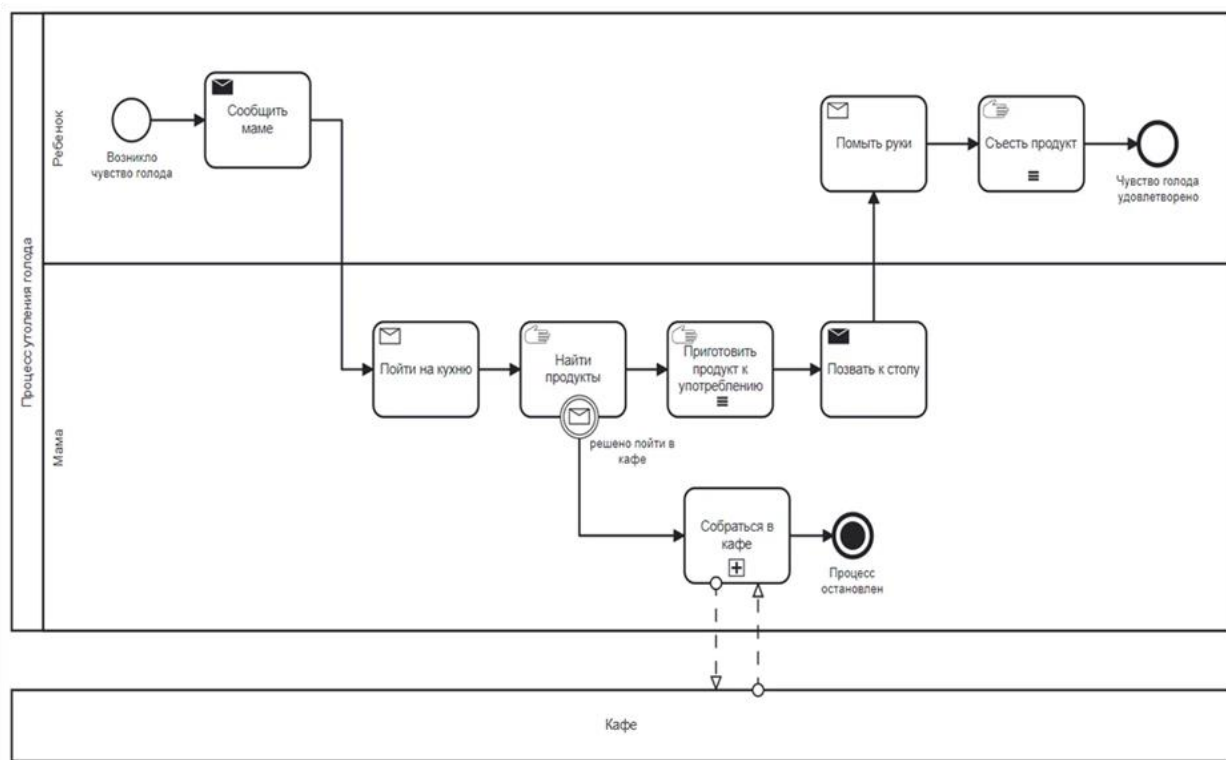
Пример. Процесс утреннего пробуждения.



Как можно видеть на диаграмме, после стартового события выполняется первое действие («Проверить время звонка»). Следующий за ним логический оператор исключаящего ИЛИ, подобно шлюзу, пропускает дальше поток управления только по одной ветке: «да» или «нет». Причём ветка «нет» здесь помечена как поток по умолчанию, который выполнится, если все остальные условия не будут верны.

После выполнения действия оператор, включающего ИЛИ (OR) пропускает поток на действие «Выпить кофе» или на действие «Узнать новости» или по обоим веткам. Исключения здесь нет, ручеёк потока управления распараллеливается на две ветки, чтобы потом объединиться снова в одну и один раз выполнить действие «приготовиться к делам». После выполнения этого действия процесс заканчивается конечным событием.

Пример. Процесс утоления голода



Стартовым событием является простое событие «Возникло чувство голода» на дорожке Ребёнок, а конечным — простое событие «Чувство голода удовлетворено» на этой же самой дорожке.

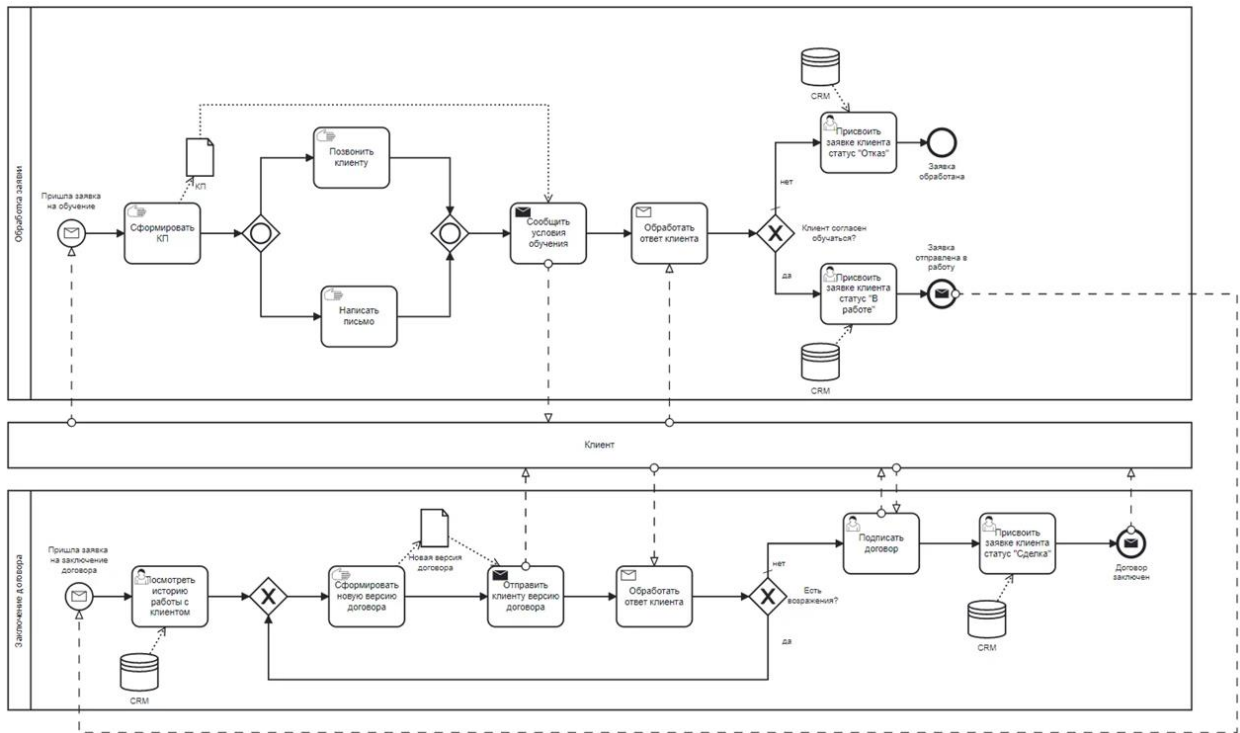
Сам процесс представлен линейным потоком, без логических ветвлений. Однако при выполнении задачи «Найти продукты» возникло граничное прерывающее событие «Решено пойти в кафе», которое запускает ветку с задачей «Собратся в кафе» и заканчивается событием-терминатором, который останавливает весь процесс в целом.

Кафе показано отдельным свёрнутым пулом, общение с которым происходит через поток сообщений в рамках свёрнутой задачи «Собратся в кафе». Предполагается, что детали выполнения задачи «Собратся в кафе» отражены на отдельной диаграмме.

Пример. Формирование клиентского предложения

Пример описания процесса обработки заявки и заключения договора продемонстрирован ниже.

Обратите внимание, что после формирования КП и создания договора, создается объект в виде файла. А для изменения статуса заявки идет обращение к базе данных. Также эта диаграмма может быть разделена на две, для простоты чтения диаграмм.

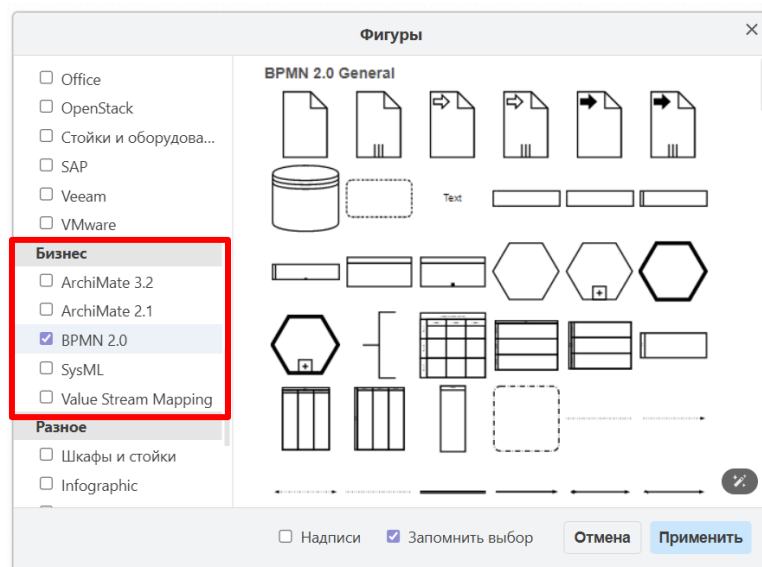


Задание и порядок проведения работы

Создать 2 диаграммы в нотации BPMN, описывающие основные бизнес-процессы в вашей системе.

Используйте только те элементы, которые приведены в данном практикуме.

Рисовать диаграммы лучше в draw.io, добавив раздел с элементами нотации BPMN.



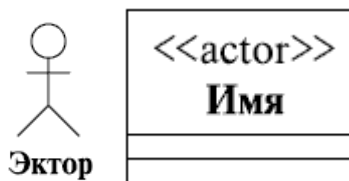
ЛАБОРАТОРНАЯ РАБОТА №5

ПОСТРОЕНИЕ ДИАГРАММЫ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ

Цель работы: освоение технологии проектирования ИС с помощью UML диаграмм

Краткие теоретические сведения

Эктор – это набор ролей, которые исполняет пользователь в ходе взаимодействия с некоторой сущностью (системой, подсистемой, классом). Эктор может быть человеком, другой системой, подсистемой или классом, которые представляют нечто за пределами рассматриваемой сущности.



Прецедент – это описание набора последовательных событий (включая возможные варианты), выполняемых системой, которые приводят к наблюдаемому эктором результату. Прецеденты описывают сервисы, предоставляемые системой экторам, с которыми она взаимодействует. Причем прецедент никогда не объясняет, "как" работает сервис, а только описывает, "что" делается.

Изображаются прецеденты в виде эллипса, внутри контура которого помещается имя (описание) прецедента. Имя прецедента обычно намного длиннее имен других элементов модели. Почему это так, в принципе, понятно: имя прецедента описывает взаимодействие эктора с системой, говорит о том, какими сообщениями они обмениваются между собой. В нашем примере с заказом обедов мы видели несколько прецедентов и наверняка читатель заметил, что имя прецедента – это, скорее, название сценария, воспроизводящегося в ходе взаимодействия эктора с системой. Причем это всегда описание с точки зрения эктора, описание услуг, предоставляемых системой пользователю. Приведем пример простейшей диаграммы, иллюстрирующей сказанное нами об обозначениях прецедента



Сценарий – это конкретная последовательность действий, иллюстрирующая поведение.

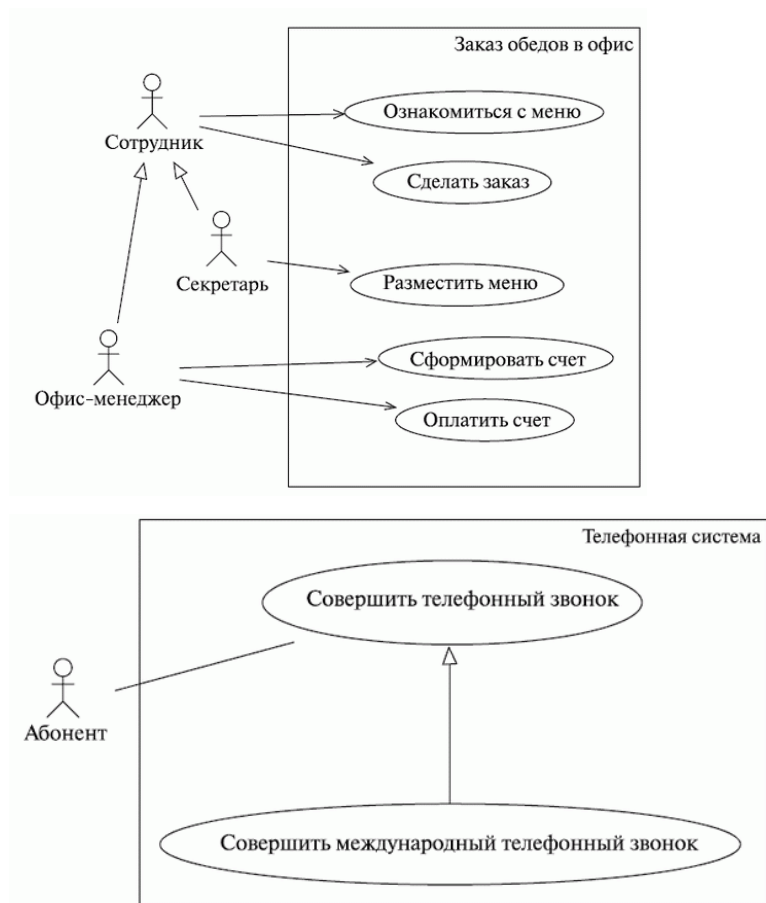
Сценарий – это повествовательный рассказ о совершаемых эктором действиях, история, эпизод, происходящий в данных временных рамках и данном контексте взаимодействия. Сценарии (в различных формах представления) широко применяются в процессе разработки программного обеспечения. Как мы уже только что отметили, написание сценария напоминает написание художественного рассказа, и этим объясняется тот факт, что использование сценариев широко распространено среди аналитиков, которые часто обладают художественными или литературными способностями. Несмотря на непрерывный повествовательный характер, сценарии можно рассматривать как последовательности действий (делать раскадровку).



Как мы уже говорили выше, обобщение (наследование) чаще всего используют между классами и интерфейсами. Однако другие элементы модели также могут находиться между собой в отношении наследования – например, пакеты (о которых мы тут не говорим), экторы, прецеденты...

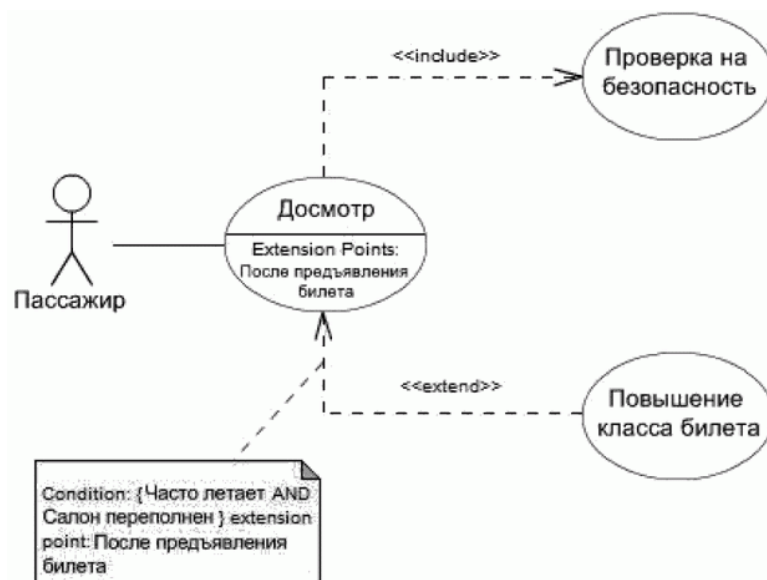
Изображается обобщение, как, конечно, помнит внимательный читатель, линией с "незакрашенной" треугольной стрелкой на конце. Обобщение – это отношение между предком и потомком, и стрелка всегда указывает на предка. Если вспомнить, что потомки наследуют (используют) свойства предка, то вполне логично вспоминается наше утверждение о том, что стрелки в UML всегда направлены в сторону того, от кого что-то требуют, чьими сервисами пользуются.

Прецедент	Действующее лицо
разместить меню	секретарь
ознакомиться с меню	сотрудник, секретарь, офис-менеджер
сделать заказ	сотрудник, секретарь, офис-менеджер
сформировать счет	офис-менеджер
оплатить счет	офис-менеджер



Отношение включения (include) означает, что в некоторой точке базового прецедента содержится поведение другого прецедента. Включаемый прецедент не существует сам по себе, а является всего лишь частью объемлющего прецедента. Таким образом, базовый прецедент как бы заимствует поведение включаемых, раскладываясь на более простые прецеденты.

Расширение (extend) дополняет прецедент другими прецедентами, "срабатывающими" при некоторых условиях, – просто добавляет в исходный прецедент последовательность действий, содержащуюся в другом прецеденте. Отношение расширения прецедента А к прецеденту В означает, что экземпляр прецедента В может включать в себя поведение, описанное в прецеденте А. Точка расширения описывается в дополнительном разделе прецедента, отделенном от его названия горизонтальной линией – точно так же, как в отдельных разделах перечисляются атрибуты класса и его операции.



Задание и порядок проведения работы

Ознакомиться с теоретическим материалом, разработать диаграмму вариантов использования в любом редакторе.

ЛАБОРАТОРНАЯ РАБОТА №6

ПОСТРОЕНИЕ ДИАГРАММЫ ПОСЛЕДОВАТЕЛЬНОСТИ

Цель работы: освоение технологии проектирования ИС с помощью UML диаграмм

Краткие теоретические сведения

Диаграммы последовательностей используются для уточнения диаграмм прецедентов, более детального описания логики сценариев использования. Это отличное средство документирования проекта с точки зрения сценариев использования.

Диаграммы последовательностей обычно содержат объекты, которые взаимодействуют в рамках сценария, сообщения, которыми они обмениваются, и возвращаемые результаты, связанные с сообщениями. Впрочем, часто возвращаемые результаты обозначают лишь в том случае, если это не очевидно из контекста.

Объекты обозначаются прямоугольниками с подчеркнутыми именами (чтобы отличить их от классов).

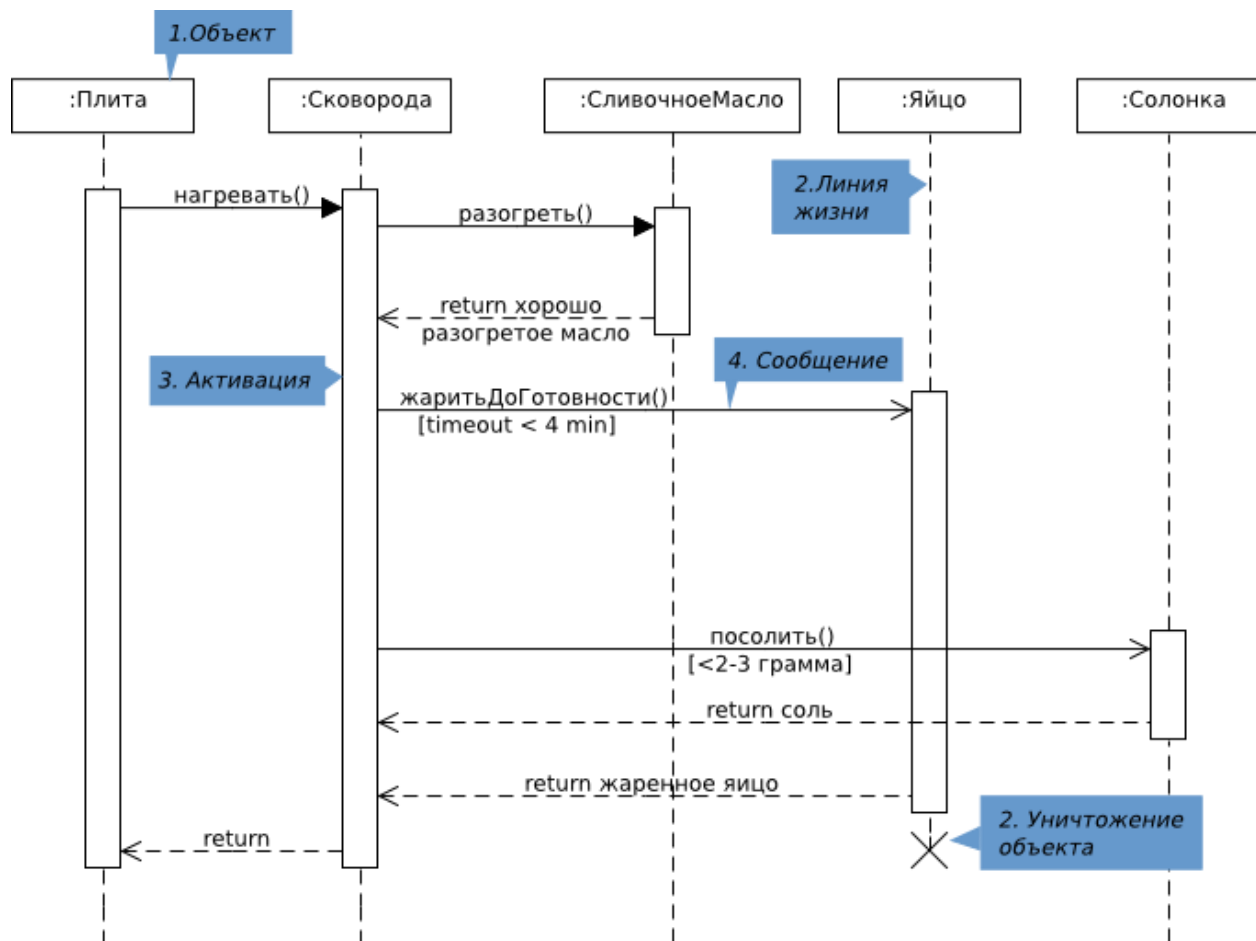
Сообщения (вызовы методов) – линиями со стрелками.

Возвращаемые результаты – пунктирными линиями со стрелками.

Прямоугольники на вертикальных линиях под каждым из объектов показывают

“время жизни” (фокус) объектов.

Диаграмма последовательности является временной диаграммой. Время в данной диаграмме течет сверху в низ



Данный вид диаграмм отражает следующие аспекты проектируемой Системы:

обмен сообщениями между объектами (в том числе в рамках обмена сообщениями со сторонними Системами)

ограничения, накладываемые на взаимодействие объектов

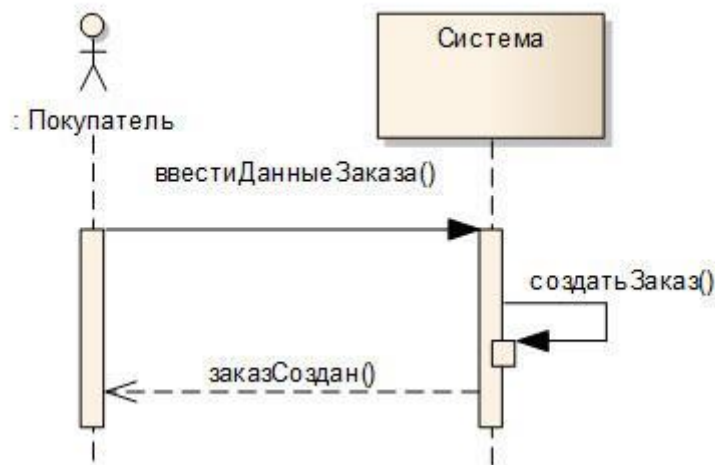
события, инициирующие взаимодействия объектов.

В отличие от диаграммы деятельности, которая показывает только последовательность (алгоритм) работы Системы, диаграммы взаимодействия акцентируют внимание разработчиков на сообщениях, инициирующих вызов определенных операций объекта (класса) или являющихся результатом выполнения операции.

Диаграмма последовательности является одной из разновидности диаграмм взаимодействия и предназначена для моделирования взаимодействия объектов Системы во времени, а также обмена сообщениями между ними.

Одним из основных принципов ООП является способ информационного обмена между элементами Системы, выражающийся в отправке и получении сообщений друг от

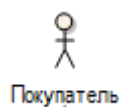
друга. Таким образом, основные понятия диаграммы последовательности связаны с понятием **Объект** и **Сообщение**.



На диаграмме последовательности объекты в основном представляю экземпляры класса или сущности, обладающие поведением. В качестве объектов могут выступать пользователи, инициирующие взаимодействие, классы, обладающие поведением в Системе или программные компоненты, а иногда и Системы в целом.

Объекты располагаются с лева на права таким образом, чтобы крайним с лева был тот объект, который инициирует взаимодействие. Неотъемлемой частью объекта на диаграмме последовательности является линия жизни объекта. Линия жизни показывает время, в течение которого объект существует в Системе. Периоды активности объекта в момент взаимодействия показываются с помощью фокуса управления. Временная шкала на диаграмме направлена сверху вниз.

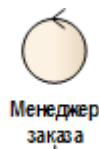
На диаграммах последовательности допустимо использование стандартных стереотипов класса:



Actor – экземпляр участника процесса (роль на диаграмме прецедентов)



Boundary – Класс-Разграничитель – используется для классов, отделяющих внутреннюю структуру системы от внешней среды (экранная форма, пользовательский интерфейс, устройство ввода-вывода). Объект со стереотипом <<boundary>> отличается от, привычного нам, класса <<Интерфейс>>, который по большей части предназначен для вызова методов класса, с которым он связан. Объект boundary показывает именно экранную форму, которая принимает и передает данные обработчику.</boundary>>

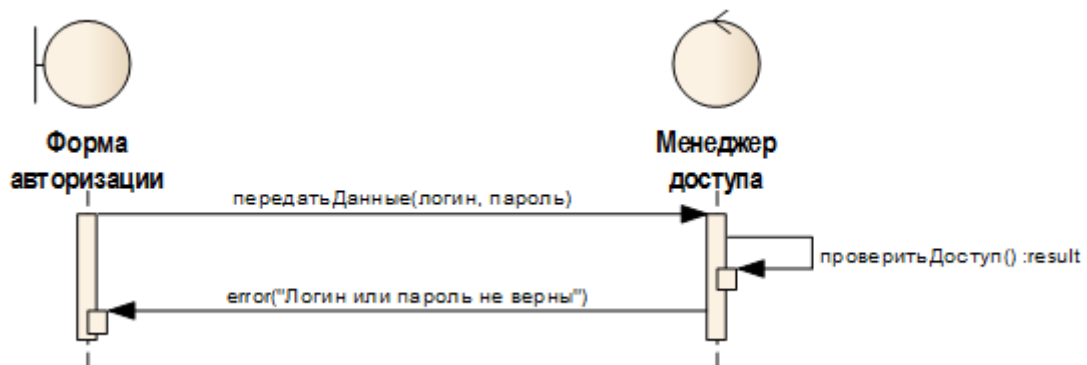


Control – Класс-контроллер – активный элемент, который используются для выполнения некоторых операций над объектами (программный компонент, модуль, обработчик)



Entity – Класс-сущность – обычно применяется для обозначения классов, которые хранят некую информацию о бизнес-объектах (соответствует таблице или элементу БД)

Также одним из основных понятий, связанных с диаграммой последовательности, является Сообщение.



На диаграмме деятельности выделяются сообщения, инициирующие ту или иную деятельность или являющиеся ее следствием. На диаграмме состояний частично показан обмен сообщениями в рамках сообщений инициирующих изменение состояния объекта.

Диаграмма последовательности объединяет диаграмму деятельности, диаграмму состояний и диаграмму классов.

Таким образом, на диаграмме последовательности мы можем увидеть следующие аспекты:

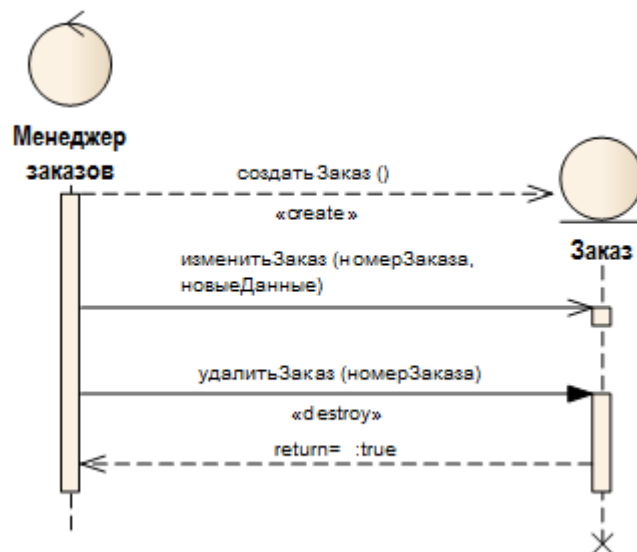
Сообщения, побуждающие объект к действию

Действия, которые вызываются сообщениями (методы) – зачастую это передача сообщения следующему объекту или возвращение определенных данных объекта

Последовательность обмена сообщениями между объектами

Итак, прием сообщения инициирует выполнение определенных действий, направленных на решение отдельной задачи тем объектом, которому это сообщение отправлено. Сообщение в большинстве случаев (за исключением диаграмм, описывающих концептуальный уровень Системы) это вызов методов отдельных объектов, поэтому для корректного исполнения метода в сообщении необходимо передать какие-то данные и определить, что мы хотим видеть в ответ. При именовании сообщения на уровне проектирования реализации системы в качестве имени сообщения следует использовать

имя метода.



В UML различают следующие виды сообщений:

синхронное сообщение (synchCall) – соответствует синхронному вызову операции и подразумевает ожидание ответа от объекта получателя. Пока ответ не поступит, никаких действий в Системе не производится.

асинхронное сообщение (asynchCall) – которое соответствует асинхронному вызову операции и подразумевает, что объект может продолжать работу, не ожидая ответа.

ответное сообщение (reply) – ответное сообщение от вызванного метода. Данный вид сообщения показывается на диаграмме по мере необходимости или, когда возвращаемые им данные несут смысловую нагрузку.

потерянное сообщение (lost) – сообщение, не имеющее адресата сообщения, т.е. для него существует событие передачи и отсутствует событие приема

найденное сообщение (found) – сообщение, не имеющее инициатора сообщения, т.е. для него существует событие приема и отсутствует событие передачи

Для сообщений также доступен ряд предопределенных стереотипов. Наиболее часто используемые стереотипы это create и destroy.

Сообщение со стереотипом create, вызывает в классе метод, которые создает экземпляр класса. На диаграмме последовательности не обязательно показывать с самого начала все объекты, участвующие во взаимодействии. При использовании сообщения со стереотипом create, создаваемый объект отображается на уровне конца сообщения.

Для уничтожения экземпляра класса используется сообщение со стереотипом destroy, при этом в конце линии жизни объекта отображаются две перекрещенные линии.

При отображении работы с сообщениями иногда возникает необходимость указать некоторые временные ограничения. Например, длительность передачи сообщения или

ожидание ответа от объекта не должно превышать определенный временной интервал. Можно указать следующие временные параметры:

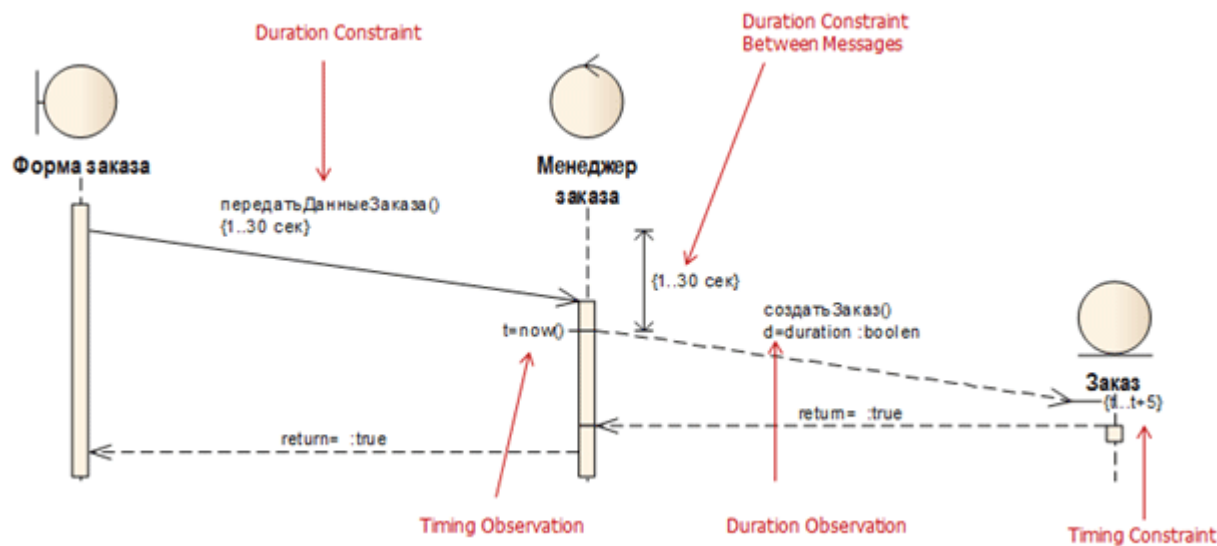
ограничение продолжительности (Duration Constraint) – минимальное и максимальное значение продолжительности передачи сообщения

ограничение продолжительности ожидания между передачей и получением сообщения (Duration Constraint Between Messages)

перехват продолжительности сообщения (Duration Observation)

временное ограничение (Timing Constraint) – временной интервал, в течение которого сообщение должно прийти к цели (устанавливается на стороне получателя)

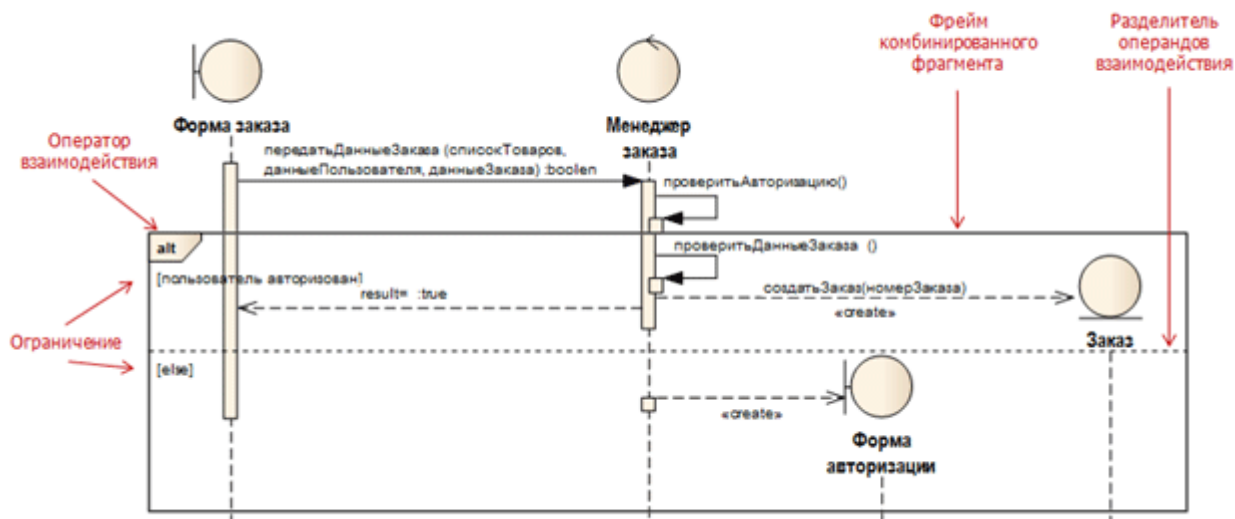
перехват времени, когда сообщение было отправлено (Timing Observation)



Форма заказа передает данные Менеджеру заказа, при этом передача данных не должна длиться больше 30 сек. – данное ограничение может понадобиться при выявлении требований к быстродействию Системы. Далее получение данных с формы инициирует запуск метода для создания экземпляра класса Заказ. Между получением данных от Формы заказа и инициализацией создания объекта должно пройти не более 30 сек., в противном случае пользователю может быть предоставлено сообщение об ошибке или недоступности сервера. Длительность передачи сообщения о создании объекта может быть зафиксирована в переменной d.

Данное значение может понадобиться при установке временного ограничения на получение ответного сообщения клиентом. В момент передачи сообщения фиксируется временное значение и заносится в переменную t. Таким образом, можно установить ограничение на стороне приемника, указав переменную t в качестве минимального значения и $t + \langle \text{допустимый интервал} \rangle$ в качестве максимального значения.

До появления UML 2.0 диаграмма последовательности рассматривалась только в рамках моделирования последовательности обмена сообщениями. Расширение сценария отображалось с помощью ветвления линий сообщений, что не давало полной картины взаимодействия объектов. Таким образом, для целей моделирования расширения сценария, параллельности процессов или цикличности использовались диаграммы деятельности. Для решения данных задач в UML 2.0 было введено понятие фрейма взаимодействия и операторов взаимодействия.



Отдельные фрагменты диаграммы взаимодействия можно выделить с помощью фрейма. Фрейм должен содержать метку оператора взаимодействия. UML содержит следующие операнды:

Alt – Несколько альтернативных фрагментов (alternative); выполняется только тот фрагмент, условие которого истинно

Opt – Необязательный (optional) фрагмент; выполняется, только если условие истинно. Эквивалентно alt с одной веткой

Par – Параллельный (parallel); все фрагменты выполняются параллельно

loop – Цикл (loop); фрагмент может выполняться несколько раз, а защита обозначает тело итерации

region – Критическая область (critical region); фрагмент может иметь только один поток, выполняющийся за один прием

Neg – Отрицательный (negative) фрагмент; обозначает неверное взаимодействие

ref – Ссылка (reference); ссылается на взаимодействие, определенное на другой диаграмме. Фрейм рисуется, чтобы охватить линии жизни, вовлеченные во взаимодействие.

Можно определять параметры и возвращать значение

Sd – Диаграмма последовательности (sequence diagram); используется для

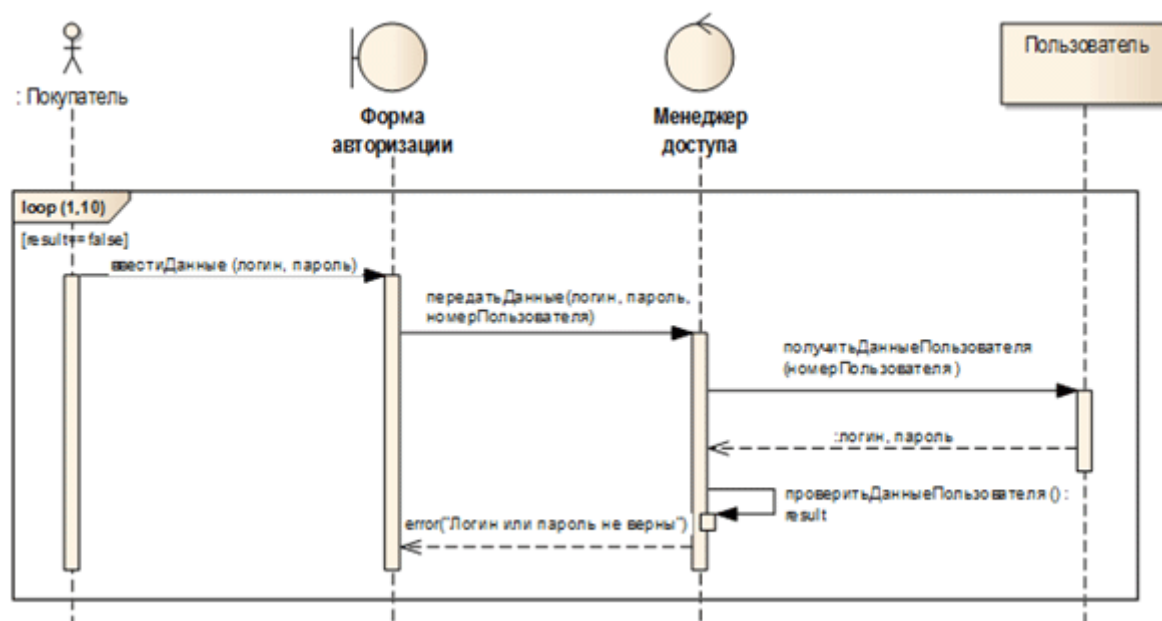
очерчивания всей диаграммы последовательности, если это необходимо.

При использовании фрагмента условного операнда фрейм должен содержать условие для ограничения взаимодействия. При использовании условного или параллельного операнда фрейм делится на регионы взаимодействия с помощью разделителя операторов взаимодействия.

К условным операндам относятся `alt` и `opt`. Операнд `alt` используется при моделировании расширения сценария, т.е. при наличии альтернативного потока взаимодействия. Оператор `opt` используется, если сообщение должно быть передано, только при истинности какого-то условия. Данный фрейм используется без деления на регионы.

Параллельность потоков взаимодействия можно изобразить с помощью операнда `par`. Внутри фрейма моделируются потоки взаимодействия в отдельных регионах.

Цикличность потока взаимодействия может быть представлена на диаграмме последовательности с помощью операнда `loop`. При использовании оператора цикла можно указать минимальное и максимальное число итераций. Также фрейм должен содержать условие, при наступлении которого взаимодействие повторяется.



Диаграммы последовательности предназначены для моделирования взаимодействия между несколькими объектами. Зачастую диаграммы последовательности создаются для моделирования взаимодействия в рамках одного прецедента.

Задание и порядок проведения работы

Ознакомиться с теоретическим материалом, разработать диаграмму

последовательности для одного из важных вариантов использования в любом редакторе.

ЛАБОРАТОРНАЯ РАБОТА №7

ПОСТРОЕНИЕ ДИАГРАММЫ АКТИВНОСТИ

Цель работы: освоение технологии проектирования ИС с помощью UML диаграмм

Краткие теоретические сведения

Диаграммы активностей (Activity Diagrams) являются представлением алгоритмов неких действий (активностей), выполняющихся в системе.

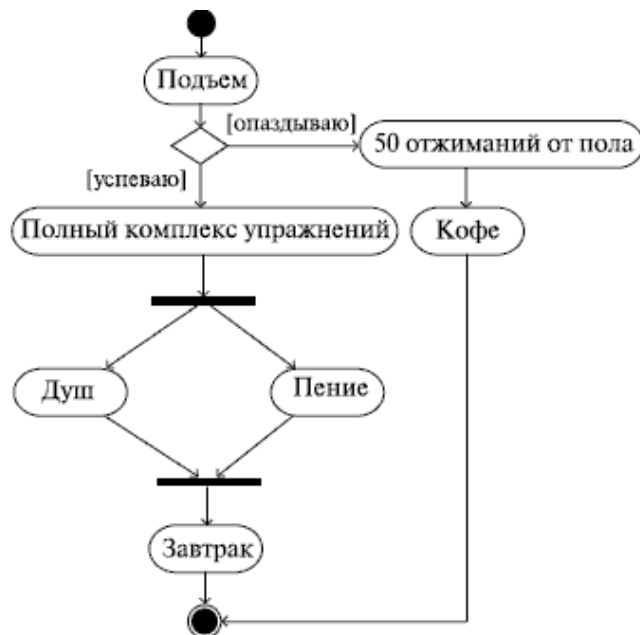
Можно построить несколько диаграмм деятельности для одной и той же системы, причем каждая из них будет фокусироваться на разных аспектах системы, показывать различные действия, выполняющиеся внутри ее. Говоря более формально, диаграммы активности, в общем-то, не имеют монополии на описание поведенческих особенностей динамических частей системы. Для этой же цели могут использоваться еще диаграммы прецедентов, последовательности, кооперации и состояний.

Именно на диаграмме деятельности представлены переходы потока управления от одной деятельности к другой. Это, по сути, разновидность диаграммы состояний, где все или большая часть состояний являются некоторыми деятельностями, а все или большая часть переходов срабатывают при завершении определенной деятельности и позволяют перейти к выполнению следующей. Как мы уже говорили (повторение – мать учения), диаграмма деятельности может быть присоединена к любому элементу модели, имеющему динамическое поведение. Кстати, исходя из вышесказанного, логичнее говорить не "диаграмма деятельности", а "диаграмма деятельностей" – во множественном числе. А еще мы предполагаем, что читатель понимает смысл понятий "деятельность", "переход" и "объект". Об объектах как об экземплярах классов мы уже говорили ранее. Понятия же деятельности (activity) как протяженного во времени составного (неатомарного) вычисления (действия, action) и перехода как передачи контроля, надеемся, понятны интуитивно, без дополнительных объяснений.

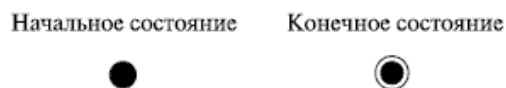
Диаграммы деятельности позволяют моделировать сложный жизненный цикл объекта, с переходами из одного состояния (деятельности) в другое. Но этот вид диаграмм может быть использован и для описания динамики совокупности объектов. Они применимы и для детализации некоторой конкретной операции, причем, как мы увидим далее, предоставляют для этого больше возможностей, чем "классическая" блок-схема. Диаграммы деятельности описывают переход от одной деятельности к другой, в отличие от

диаграмм взаимодействия, где акцент делается на переходах потока управления от объекта к объекту.

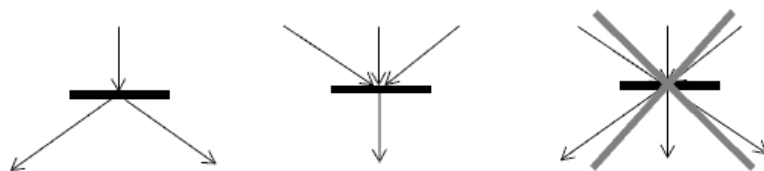
Как говорится, лучше один раз увидеть, чем сто раз услышать. Мы достаточно разрекламировали диаграммы деятельности. Пора взглянуть на пример.



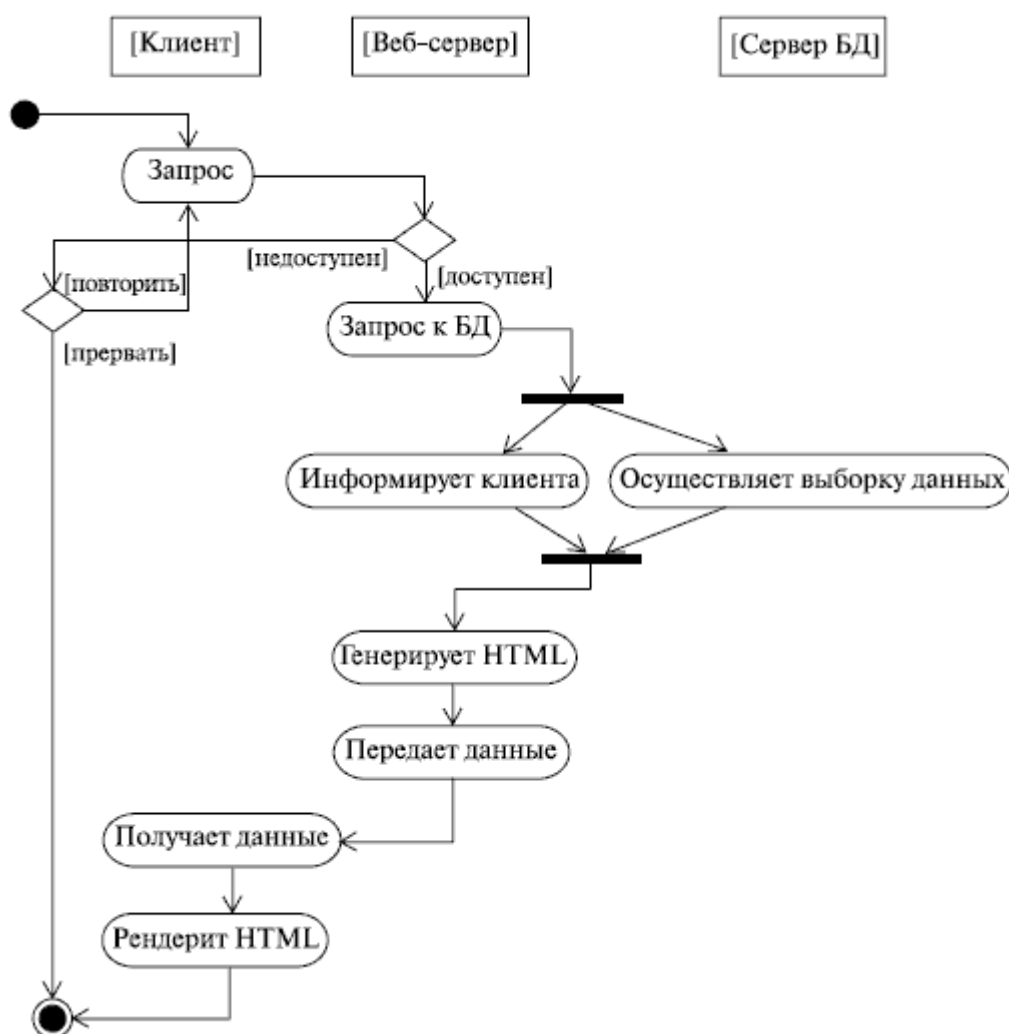
Эта диаграмма довольно точно описывает ежедневную последовательность действий автора этих строк (до момента ухода на работу). Как видим, все очень просто и понятно. Действия показаны скругленными прямоугольниками, как в блок-схеме, — мы узнаем даже ромбик символа принятия решения с обозначениями условий возле переходов. Да, отличия от блок-схемы не так уж сильны. Более того, эти отличия выглядят как логичное расширение нотации блок-схем. Обратим внимание на то, что начало и конец уже не изображаются одинаковым безликим кружком. Начало теперь закрашено, а конец изображен в виде символа, напоминающего кошачий глаз (кстати, это образное название — "кошачий глаз" — уже намертво въелось в жаргон архитекторов и аналитиков).



Без пояснений понятен также смысл символа, предшествующего принятию душа и пению и следующего за ними — он означает распараллеливание, а затем опять слияние воедино (синхронизацию) потоков управления, т. е. операции "пение" и "душ" выполняются одновременно. Нотация проста: несколько потоков управления сливаются в один или один поток разделяется на несколько. Третьего не дано.



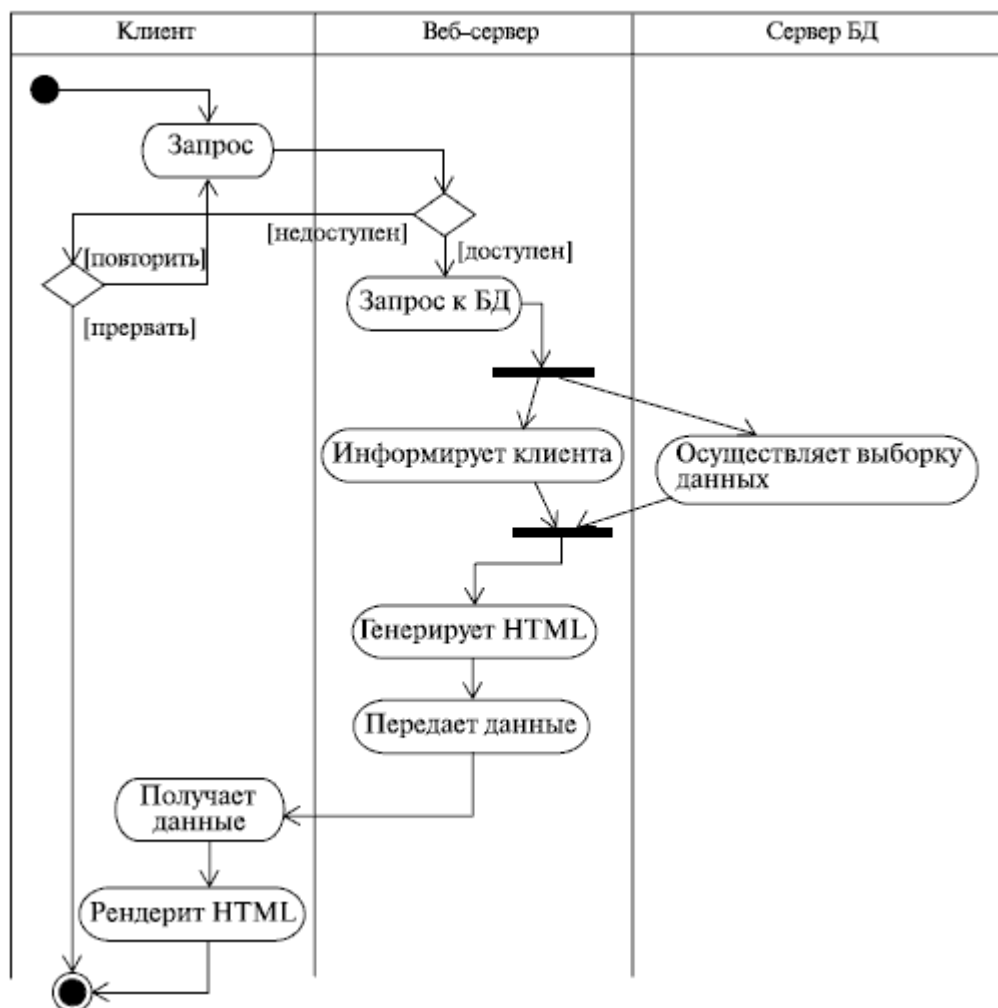
Конечно, это не единственные отличия диаграммы активностей от блок-схемы. На диаграмме деятельности можно не только показать параллельно выполняемые действия, но и указать состояния объектов (так же, как и на представлениях конечных автоматов, о которых нам так много говорили в университетах), также есть возможность показывать распределение ролей и т. д. Вот еще пример, подтверждающий, что диаграмма активностей – это нечто большее, чем блок-схема.



Смысл диаграммы вполне понятен и без дополнительных объяснений. Как вы уже, конечно, догадались, на ней показана работа с веб-приложением, которое решает некую задачу в удаленной базе данных. Привлекает внимание странное расположение активностей на этой диаграмме: они как бы разбросаны по трем беговым дорожкам, каждая из которых соответствует поведению одного из трех объектов – клиента, веб-сервера и сервера баз данных. Благодаря этому легко определить, каким из объектов выполняется

каждая из активностей, и неожиданно приходит понимание того, что "странность" этой диаграммы, оказывается, очень упрощает ее восприятие.

Аналогия с дорожками действительно очень удачна. Именно таково официальное название элемента нотации UML, позволяющего указать распределение ролей на диаграмме активностей. Только дорожки это не беговые, а плавательные – они так и называются: swimlanes. Более формально, дорожка – часть области диаграммы деятельности, на которой отображаются только те деятельности, за которые отвечает конкретный объект.

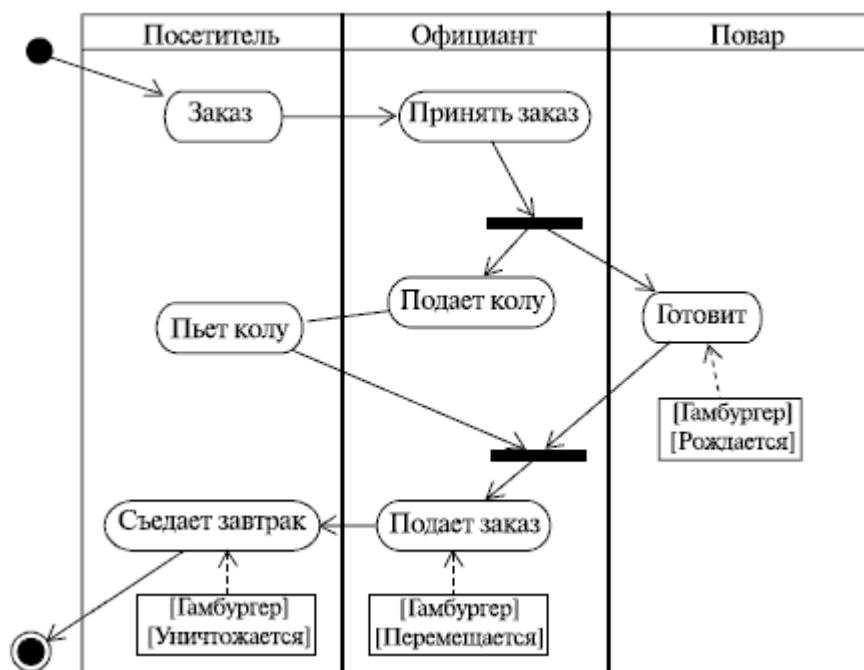


Предназначены они для разбиения диаграммы в соответствии с распределением ответственности за действия. Имя дорожки может означать роль или объект, которому она соответствует. При использовании дорожек нотация слегка изменяется. Вот как, к примеру, выглядит диаграмма из предыдущего примера, перерисованная с использованием дорожек.

Кстати, дорожки могут быть не только вертикальными, но и, если вам как автору так удобнее, горизонтальными. Изображаются горизонтальные дорожки аналогично – просто поверните "обычные" дорожки на 90 градусов против часовой стрелки!

Есть еще один нюанс нотации диаграмм активностей, о котором мы пока не

говорили: это так называемая траектория объекта, или поток объекта (object flow). Суть его состоит в том, что на диаграмме деятельности можно изобразить и объекты, относящиеся к деятельности. С помощью символа зависимости (пунктирная стрелка, помните?) эти объекты можно соотнести с той деятельностью или переходом, где они создаются, изменяются или уничтожаются. Представим такую ситуацию из повседневной жизни: вы приходите в какой-нибудь фастфуд и заказываете гамбургер с колой. Что, знакомо? Во время приготовления завтрака повар создает новый объект – гамбургер. Пока вы нетерпеливо выпиваете колу, официант перемещает этот объект (подает ваш заказ). Естественно, во время завтрака вы уничтожаете этот объект. Вот как это выглядит на диаграмме .



На этом можно было бы и закончить наш разговор о нотации диаграмм активностей и их отличиях от блок-схем. Если бы не одно НО. Мы говорили, что деятельность – это протяженное по времени составное действие. Составное! То есть составленное из более простых действий. Вот эти-то самые простые (атомарные) действия, а вернее, последовательность их выполнения, частенько изображают внутри деятельности в виде маленькой диаграммы активностей. Это слегка напоминает матрешку – одна (а часто и не одна) диаграмма внутри другой. Мы не будем долго говорить об этом: нашей целью было просто обратить внимание читателя на подобную возможность "вложенных" диаграмм. Мы просто покажем пример, позаимствованный нами из Zicom Mentor

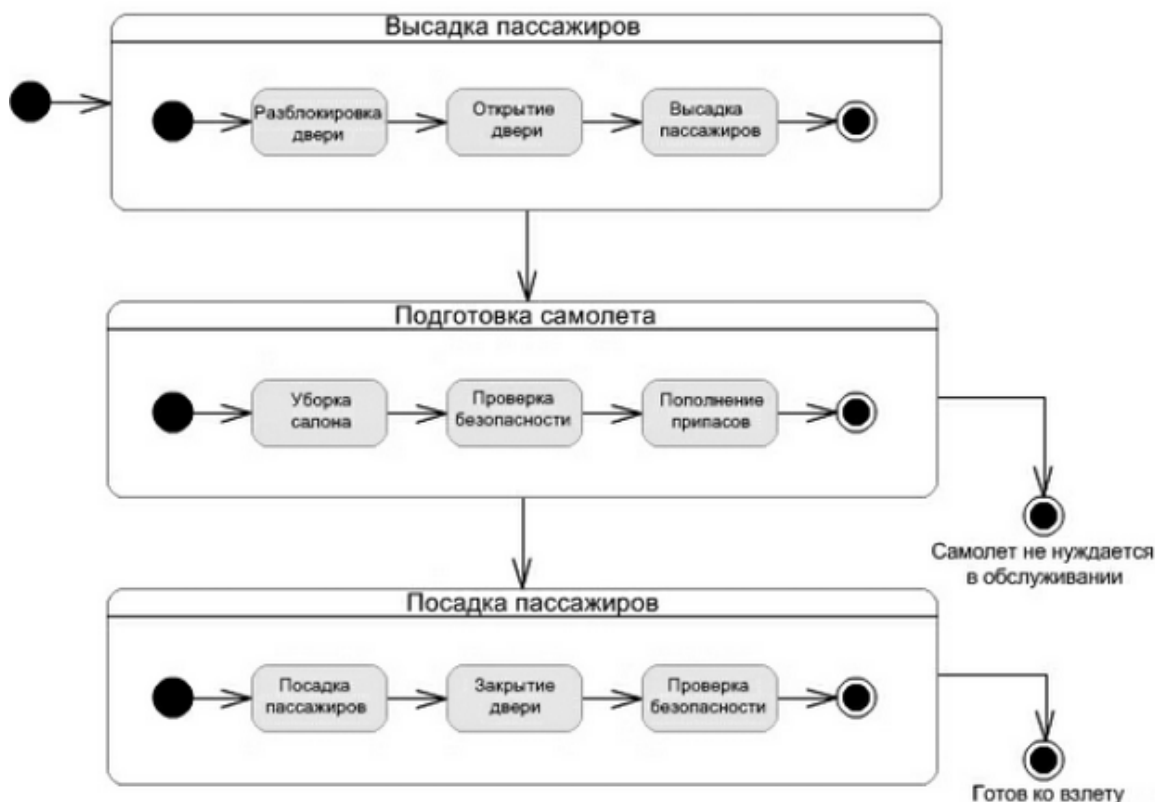


Диаграмма описывает высадку пассажиров самолета, достигших пункта назначения, и посадку новых пассажиров. Предлагаем читателю самому внимательно рассмотреть эту диаграмму. Из нее, например, можно почерпнуть, что конечных состояний может быть больше одного. Кстати, кроме начального и конечного состояний есть еще конечное состояние потока (Flow final mode). От конечного состояния оно отличается вот чем: конечное состояние потока означает завершение одного потока управления, а конечное состояние говорит о завершении всех потоков управления внутри деятельности. Обозначается конечное состояние потока простым символом, напоминающим лампочку накаливания в схемах электрических цепей

Начальное состояние



Конечное состояние



Конечное состояние потока



Задание и порядок проведения работы

Ознакомиться с теоретическим материалом, разработать диаграмму активности для одного из важных вариантов использования в любом редакторе. Вариант использования не должен совпадать с тем, что выбран в ЛР 6.

ЛАБОРАТОРНАЯ РАБОТА №8

ПРОЕКТИРОВАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

Цель работы: приобрести навыки проектирования графического интерфейса пользователя.

Краткие теоретические сведения

Графический интерфейс пользователя (GUI) – разновидность пользовательского интерфейса, в котором элементы интерфейса (меню, кнопки, значки, списки и т. п.), представленные пользователю на дисплее, исполнены в виде графических изображений.

В GUI пользователь имеет произвольный доступ (с помощью устройств ввода – клавиатуры, мыши, джойстика и т. п.) ко всем видимым экранным объектам (элементам интерфейса) и осуществляет непосредственное манипулирование ими.

Графический интерфейс пользователя является частью пользовательского интерфейса и определяет взаимодействие с пользователем на уровне визуализированной информации.

Можно выделить следующие виды графического интерфейса пользователя:

- простой: типовые экранные формы и стандартные элементы интерфейса, обеспечиваемые самой подсистемой GUI;
- истинно-графический, двухмерный: нестандартные элементы интерфейса и оригинальные метафоры, реализованные собственными средствами приложения или сторонней библиотекой;
- трёхмерный.

Проектирование графического интерфейса пользователя представляет собой междисциплинарную деятельность. Оно требует усилий многофункциональной бригады – один человек, как правило, не обладает знаниями, необходимыми для реализации многоаспектного подхода к проектированию GUI-интерфейса. Надлежащее проектирование GUI-интерфейса требует объединения навыков художника-графика, специалиста по анализу требований, системного проектировщика, программиста, эксперта по технологии, специалиста в области социальной психологии, а также, возможно, некоторых других специалистов, в зависимости от характера системы.

В современном мире миллиарды вычислительных устройств. Еще больше программ для них. И у каждой свой интерфейс, являющийся «рычагами» взаимодействия между пользователем и машинным кодом. Не удивительно, что чем лучше интерфейс, тем

эффективнее взаимодействие.

Однако далеко не все разработчики и даже дизайнеры, задумываются о создании удобного и понятного графического интерфейса пользователя.

Какие элементы интерфейса (ЭИ) создавать?

Разработка интерфейса обычно начинается с определения задачи или набора задач, для которых продукт предназначен.

Простое должно оставаться простым. Не стоит усложнять интерфейсы. Нужно постоянно думать о том, как сделать интерфейс проще и понятнее.

Пользователи не задумываются над тем, как устроена программа. Все, что они видят – это интерфейс. Поэтому, с точки зрения потребителя именно интерфейс является конечным продуктом.

Интерфейс должен быть ориентированным на человека, т.е. отвечать нуждам человека и учитывать его слабости. Нужно постоянно думать о том, с какими трудностями может столкнуться пользователь.

Необходимо думать о поведении и привычках пользователей. Не менять хорошо известные всем ЭИ на неожиданные, а новые делать интуитивно понятными.

Разрабатывать интерфейс необходимо исходя из принципа наименьшего возможного количества действий со стороны пользователя.

Какой должен быть дизайн элементов интерфейса?

В дизайне ЭИ нужно учитывать все: начиная от цвета, формы, пропорций, заканчивая когнитивной психологией. Однако, несколько принципов все же стоит отметить:

- Цвет. Цвета делятся на теплые (желтый, оранжевый, красный), холодные (синий, зеленый), нейтральные (серый). Обычно для ЭИ используют теплые цвета. Это как раз связано с психологией восприятия. Стоит отметить, что мнение о цвете – очень субъективно и может меняться даже от настроения пользователя.

- Форма. В большинстве случаев – прямоугольник со скругленными углами. Или круг. Опять же, форма как и цвет достаточно субъективна.

- Основные ЭИ (часто используемые) должны быть выделены. Например, размером или цветом.

- Иконки в программе должны быть очевидными. Или подписанными. Ведь, по сути дела, вместо того чтобы объяснять, пиктограммы зачастую сами требуют для себя объяснений.

Как правильно расположить элементы интерфейса на экране?

Есть утверждение, что визуальная привлекательность основана на пропорциях. Помните известное число 1.62? Это так называемый принцип Золотого сечения. Суть в том, что весь отрезок относится к большей его части так, как большая часть, относится к меньшей. Например, общая ширина сайта 900px, делим 900 на 1.62, получаем ~555px, это ширина блока с контентом. Теперь от 900 отнимаем 555 и получаем 345px. Это ширина меньшей части.

Перед расположением, ЭИ следует упорядочить (сгруппировать) по значимости. Т.е. определить, какие наиболее важны, а какие – менее.

Обычно (но не обязательно), элементы размещаются в следующей градации: слева направо, сверху вниз. Слева вверху самые значимые элементы, справа внизу – менее. Это связано с порядком чтения текста. В случае с сенсорными экранами, самые важные элементы, располагаются в области действия больших пальцев рук.

Необходимо учитывать привычки пользователя. Например, если в Windows кнопка закрыть находится в правом верхнем углу, то программе аналогичную кнопку необходимо расположить там же. Т.е. интерфейс должен иметь как можно больше аналогий, с известными пользователю вещами.

Размещать ЭИ стоит поближе там, где большую часть времени находится курсор пользователя. Что бы ему не пришлось перемещать курсор, например, от одного конца экрана к другому.

Элемент интерфейса можно считать видимым, если он либо в данный момент доступен для органов восприятия человека, либо он был настолько недавно воспринят, что еще не успел выйти из кратковременной памяти. Для нормальной работы интерфейса, должны быть видимы только необходимые вещи – те, что идентифицируют части работающих систем, и те, что отображают способ, которым пользователь может взаимодействовать с устройством.

Отступы между ЭИ лучше делать равными или кратными друг-другу.

Как элементы интерфейса должны себя вести?

Пользователи привыкают. Например, при удалении файла, появляется окно с подтверждением: «Да» или «Нет». Со временем, пользователь перестает читать предупреждение и по привычке нажимает «Да». Поэтому диалоговое окно, которое было призвано обеспечить безопасность, абсолютно не выполняет своей роли. Следовательно, необходимо дать пользователю возможность отменять, сделанные им действия.

Если пользователю дают информацию, которую он должен куда-то ввести или как-то обработать, то информация должна оставаться на экране до того момента, пока человек ее не обработает. Иначе он может просто забыть.

Нужно избегать двусмысленности. Например, на фонарике есть одна кнопка. По нажатию фонарик включается, нажали еще раз – выключился. Если в фонарике перегорела лампочка, то при нажатии на кнопку не понятно, включаем мы его или нет. Поэтому, вместо одной кнопки выключателя, лучше использовать переключатель (например, checkbox с двумя позициями: «вкл.» и «выкл.»). За исключением случаев, когда состояние задачи, очевидно.

Имеет смысл делать монотонные интерфейсы. Монотонный интерфейс – это интерфейс, в котором какое-то действие, можно сделать только одним способом. Такой подход обеспечит быструю привыкаемость к программе и автоматизацию действий.

Не стоит делать адаптивные интерфейсы, которые изменяются со временем. Так как для выполнения какой-то задачи, лучше изучать только один интерфейс, а не несколько. Пример – стартовая страница браузера Chrome.

Если задержки в процессе выполнения программы неизбежны или действие производимое пользователем очень значимо, важно, чтобы в интерфейсе была предусмотрена сообщающая о них обратная связь. Например, можно использовать индикатор хода выполнения задачи (status bar).

ЭИ должны отвечать. Если пользователь произвел клик, то ЭИ должен как-то отозваться, чтобы человек понял, что клик произошел.

Карта навигации – информация на карте навигации аналогична разделу «Содержание» обычной книги. В карте представлен полный перечень разделов и/или всех страниц, имеющихся на сайте. Нередко, заголовки страниц в списке служат ссылками на эти страницы. Пример:



Задание и порядок проведения работы

1. Создайте карту навигации для выбранной системы. На карте в зависимости от специфики системы выделите разделы, доступные различным пользователям в зависимости от роли, опишите условия перехода из различных разделов (при необходимости)

2. Подберите для графического интерфейса основную палитру (3-5 цветов), укажите их.

3. Используя графический редактор на выбор, создайте основные (2-3) макеты графического интерфейса пользователя. Предлагаемые системы:

Pencil – <http://pencil.evolus.vn/>

Moqups – <https://moqups.com/>

Figma

ЛАБОРАТОРНАЯ РАБОТА №9

ПРОЕКТИРОВАНИЕ БД

Цель работы: освоение технологии проектирования баз данных.

Краткие теоретические сведения

Этапы проектирования баз данных

1. Концептуальное проектирование – сбор, анализ и редактирование требований к данным. По окончании данного этапа получаем концептуальную модель, независимую от структуры базы данных. Часто она представляется в виде модели "сущность-связь".

2. Логическое проектирование – преобразование требований к данным в структуры данных. На выходе получаем СУБД-ориентированную структуру базы данных и спецификации прикладных программ. На этом этапе часто моделируют базы данных применительно к различным СУБД и проводят сравнительный анализ моделей.

3. Физическое проектирование – определение особенностей хранения данных, методов доступа и т.д.

Модель «Сущность – связь»

Одной из распространенных моделей концептуальной схемы является модель «сущность-связь» (ERD).

Под сущностью понимают основное содержание объекта предметной области , о

котором собирают информацию. В качестве сущности могут выступать место, вещь, личность, явление.

Экземпляр сущности – конкретный объект.

Сущность принято определять атрибутами – поименованными характеристиками.

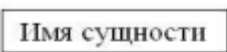
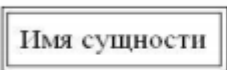
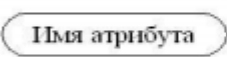
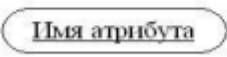
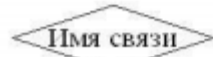
Например: сущность – студент, атрибуты – ФИО, год рождения, адрес, номер группы и т.д.

Нотации для ERD

1. Нотация IDEF1X. Её относят к фундаментальным, но на практике давно не используют, потому что есть более удобные варианты.

2. Нотация Чена. Классическая нотация, которая состоит из простых символов – прямоугольников, овалов и линий. Из-за этого нотацию часто используют для концептуальных моделей, которые презентуют заказчику. Человеку, который далёк от аналитики данных, проще разобраться в понятных диаграммах со знакомыми символами.

Элементы нотации Чена:

Обозначение	Описание
 Имя сущности	Набор независимых сущностей
 Имя сущности	Набор зависимых сущностей
 Имя атрибута	Значение атрибута
 Имя атрибута	Ключевой атрибут
 Имя связи	Набор связей

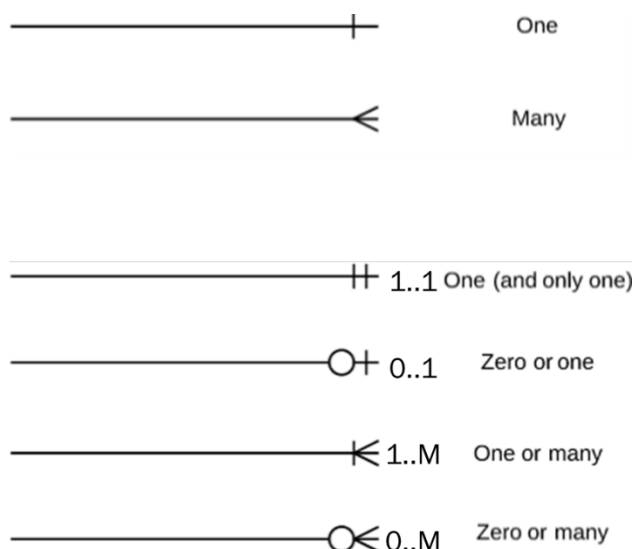
3. Нотация Мартина. Её ещё называют «воронья лапка» (от англ. Crow's Foot). Она компактнее нотации Чена, поэтому её используют для построения ER-моделей логического уровня, когда нужно описать в модели все атрибуты сущностей.

Элементы нотации Мартина:

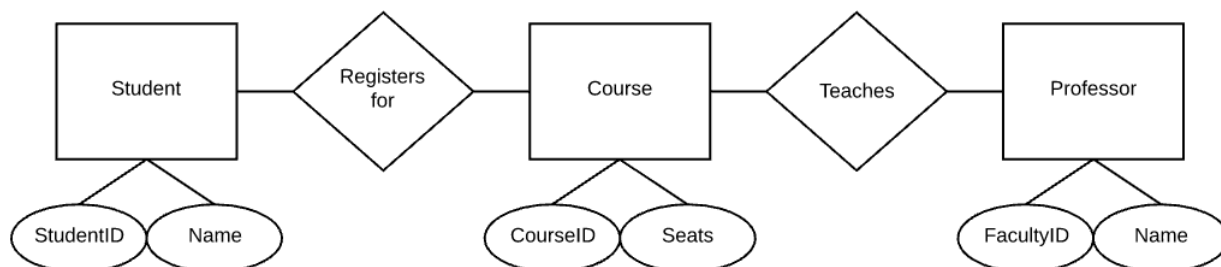


Связи в нотации представлены в двух вариантах – без указания

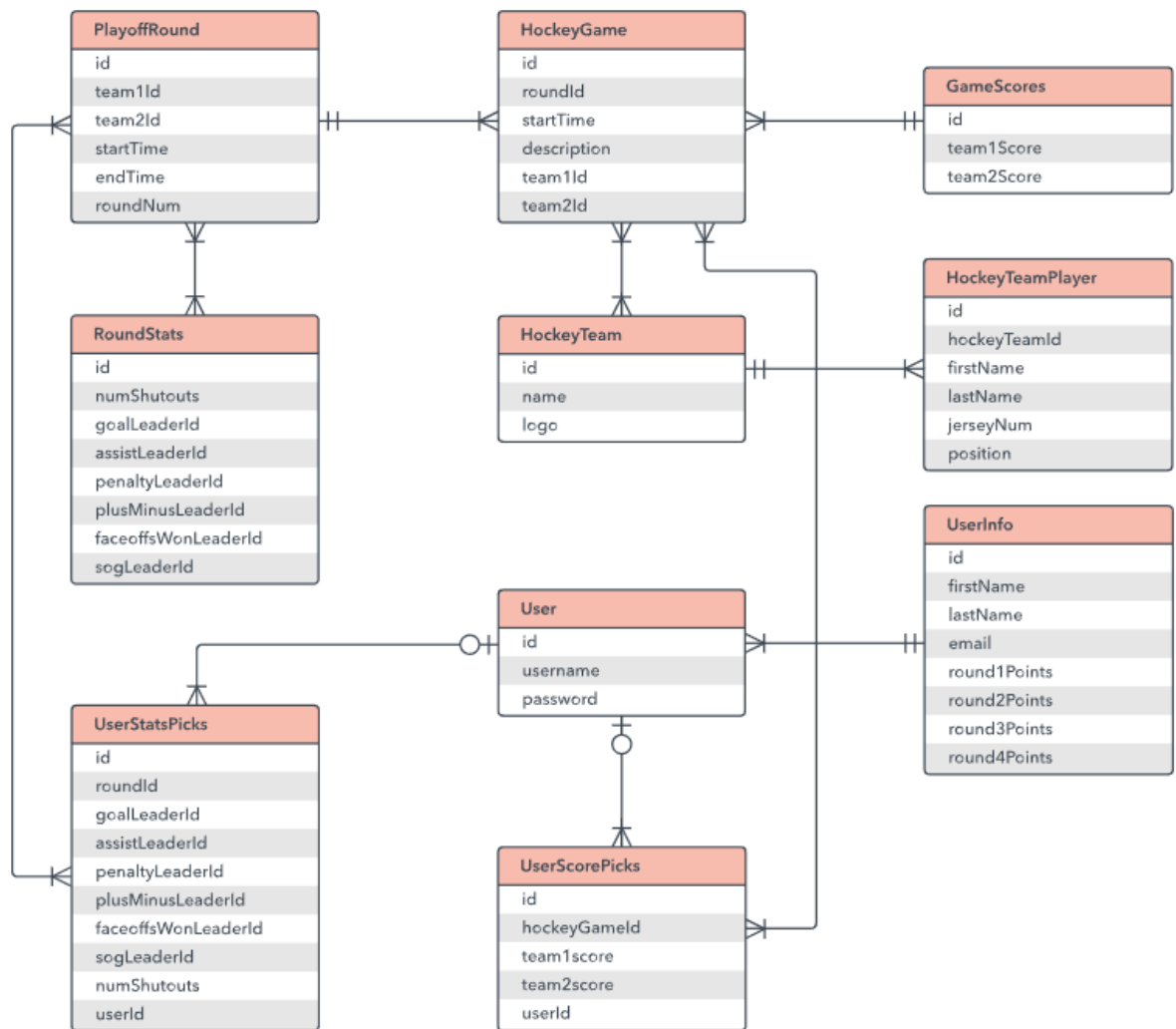
обязательности/необязательности (первые 2) и с указанием (следующие 4).



– Концептуальная модель включает описания объектов и их взаимосвязей, представляющих интерес в рассматриваемой предметной области (ПО) и выявляемых в результате анализа данных. Концептуальная модель применяется для структурирования предметной области с учетом интересов пользователей системы. Она дает возможность систематизировать информационное содержание предметной области, позволяет как бы "подняться вверх" над ПО и увидеть ее отдельные элементы. При этом, уровень детализации зависит от выбранной модели. Пример концептуальной ERD в нотации Чена:



– Логическое проектирование – моделирование построенной информационной системы и проектирование её отдельных составляющих в форме, соответствующей реальной базе данных. В процессе логического проектирования требования к данным преобразуются в структуры используемой СУБД. Пример логической ERD в нотации Мартина:



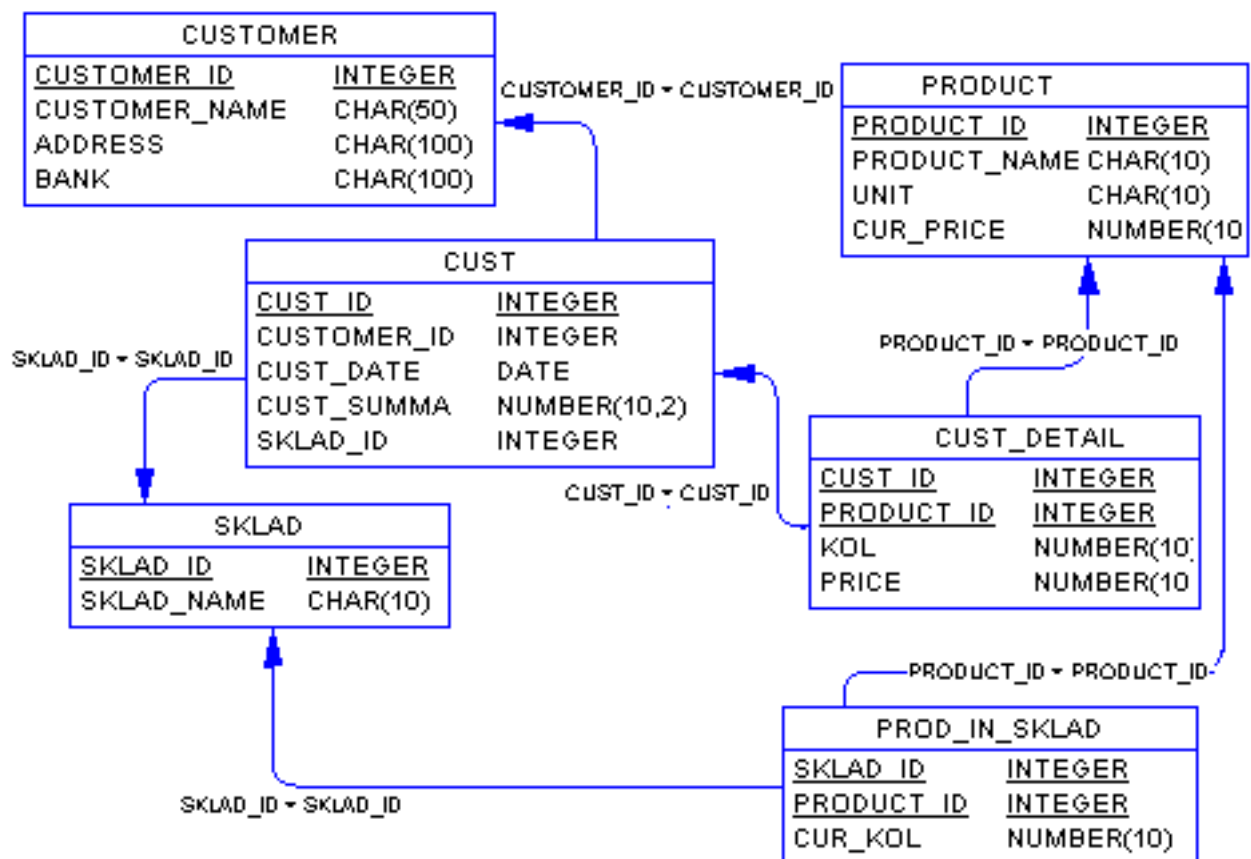
– Физическое проектирование заключается в увязке логической структуры БД и физической среды хранения с целью наиболее эффективного размещения данных, т.е. отображение логической структуры БД в структуру хранения.

При этом решения, принятые на уровне логического моделирования определяют некоторые границы, в пределах которых можно развивать физическую модель данных. Точно также, в пределах этих границ можно принимать различные решения.

Физическая модель данных полностью зависит от конкретной СУБД, фактически являясь отображением системного каталога. В физической модели содержится информация о всех объектах БД.

Следовательно, одной и той же логической модели могут соответствовать несколько разных физических моделей. Если в логической модели не имеет значения, какой конкретно тип данных имеет атрибут, то в физической модели важно описать всю информацию о конкретных физических объектах – таблицах, колонках, индексах, процедурах и т. д.

Пример физической ERD (На физическом уровне чаще всего «вороньи лапки» заменяются на стрелки 1 -> M)



Нормализация БД

Нормальная форма — требование, предъявляемое к структуре таблиц в теории реляционных баз данных для устранения из базы избыточных функциональных зависимостей между атрибутами (полями таблиц).

Метод нормальных форм (НФ) состоит в сборе информации о объектах решения задачи в рамках одного отношения и последующей декомпозиции этого отношения на несколько взаимосвязанных отношений на основе процедур нормализации отношений.

Цель нормализации: исключить избыточное дублирование данных, которое является причиной аномалий, возникших при добавлении, редактировании и удалении кортежей (строк таблицы).

- 1) Требования первой нормальной формы (1NF)
 - a) В таблице не должно быть дублирующих строк
 - b) В каждой ячейке таблицы хранится атомарное значение (одно не составное значение)
 - c) В столбце хранятся данные одного типа
 - d) Отсутствуют массивы и списки в любом виде
- 2) Требования второй нормальной формы (2NF)
 - a) Таблица должна находиться в первой нормальной форме
 - b) Таблица должна иметь ключ

- с) Все неключевые столбцы таблицы должны зависеть от полного ключа (в случае если он составной)
- 3) Требования третьей нормальной формы (3NF)
 - а) Требование третьей нормальной формы (3NF) заключается в том, чтобы в таблицах отсутствовала транзитивная зависимость.
- 4) Требования нормальной формы Бойса-Кодда
 - а) Таблица должна находиться в третьей нормальной форме. Здесь все как обычно, т.е. как и у всех остальных нормальных форм, первое требование заключается в том, чтобы таблица находилась в предыдущей нормальной форме, в данном случае в третьей нормальной форме;
 - б) Ключевые атрибуты составного ключа не должны зависеть от неключевых атрибутов.

Задание и порядок проведения работы

Опираясь на анализ предметной области по вашей теме, создайте концептуальную, логическую и физическую модели вашей БД.